Lecture 10: Kernel Methods and Support Vector Machines

Tao LIN

SoE, Westlake University

November 10, 2026



1 Quick Review and Today's Questions

2 Support Vector Machines (SVM)

3 Kernel Methods and the Kernel Trick

Quick Review: How Have We Built Predictors?

- **Linear models:** learn a weighted sum of input features. Simple to fit; in two dimensions, the decision boundary is a straight line.

Quick Review: How Have We Built Predictors?

- **Linear models:** learn a weighted sum of input features. Simple to fit; in two dimensions, the decision boundary is a straight line.
- **Last lecture:** KNN predicts from nearby examples; trees partition the input space; forests combine randomized trees.

Quick Review: How Have We Built Predictors?

- **Linear models:** learn a weighted sum of input features. Simple to fit; in two dimensions, the decision boundary is a straight line.
- **Last lecture:** KNN predicts from nearby examples; trees partition the input space; forests combine randomized trees.
- **A shared lesson:** the representation and the notion of similarity determine which patterns a model can capture.

Quick Review: How Have We Built Predictors?

- **Linear models:** learn a weighted sum of input features. Simple to fit; in two dimensions, the decision boundary is a straight line.
- **Last lecture:** KNN predicts from nearby examples; trees partition the input space; forests combine randomized trees.
- **A shared lesson:** the representation and the notion of similarity determine which patterns a model can capture.

Today's questions If several lines separate the two classes, which should we choose?

Can we learn nonlinear boundaries using inner products?

From Prediction to Correctness

A linear classifier first produces a score, then predicts its sign:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \operatorname{sign}(f(\mathbf{x})), \qquad y \in \{-1, +1\}.$$

Does a positive score always mean a correct prediction?

From Prediction to Correctness

A linear classifier first produces a score, then predicts its sign:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \operatorname{sign}(f(\mathbf{x})), \qquad y \in \{-1, +1\}.$$

Does a positive score always mean a correct prediction?

True label	Correct when	How to read the score
$y = +1$	$f(\mathbf{x}) > 0$	A positive score is good.
$y = -1$	$f(\mathbf{x}) < 0$	A negative score is good.

From Prediction to Correctness

A linear classifier first produces a score, then predicts its sign:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \operatorname{sign}(f(\mathbf{x})), \qquad y \in \{-1, +1\}.$$

Does a positive score always mean a correct prediction?

True label	Correct when	How to read the score
$y = +1$	$f(\mathbf{x}) > 0$	A positive score is good.
$y = -1$	$f(\mathbf{x}) < 0$	A negative score is good.

One rule for both classes Multiply by the true label: $m = yf(\mathbf{x})$.

$m > 0$: correct $m < 0$: wrong $m = 0$: on the boundary.

This label-adjusted score is called the functional margin.

From Prediction to Correctness

A linear classifier first produces a score, then predicts its sign:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \operatorname{sign}(f(\mathbf{x})), \qquad y \in \{-1, +1\}.$$

Does a positive score always mean a correct prediction?

True label	Correct when	How to read the score
$y = +1$	$f(\mathbf{x}) > 0$	A positive score is good.
$y = -1$	$f(\mathbf{x}) < 0$	A negative score is good.

One rule for both classes Multiply by the true label: $m = yf(\mathbf{x})$.

$m > 0$: correct $m < 0$: wrong $m = 0$: on the boundary.

This label-adjusted score is called the functional margin.

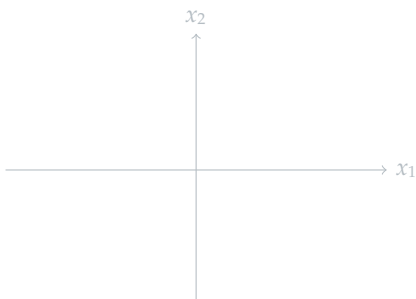
Same score, different labels: $f(\mathbf{x}) = -2$ gives $m = +2$ if $y = -1$, but $m = -2$ if $y = +1$.

Where Does the Prediction Change?

We just used the sign of a score to predict a class. Now view this rule in input space.

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \operatorname{sign}(f(\mathbf{x})).$$

Where does the predicted class change?



Where Does the Prediction Change?

We just used the sign of a score to predict a class. Now view this rule in input space.

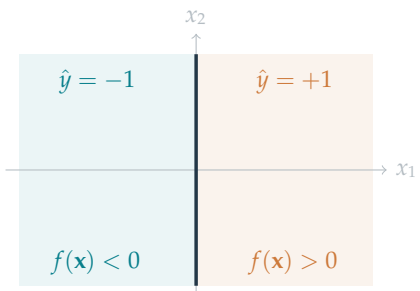
$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \operatorname{sign}(f(\mathbf{x})).$$

Where does the predicted class change?

The sign changes as the score crosses zero:

$$f(\mathbf{x}) = 0 \quad \Longleftrightarrow \quad \mathbf{w}^\top \mathbf{x} + b = 0.$$

This is the decision boundary.



Example: $f(\mathbf{x}) = x_1$, boundary $x_1 = 0$.

Where Does the Prediction Change?

We just used the sign of a score to predict a class. Now view this rule in input space.

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \operatorname{sign}(f(\mathbf{x})).$$

Where does the predicted class change?

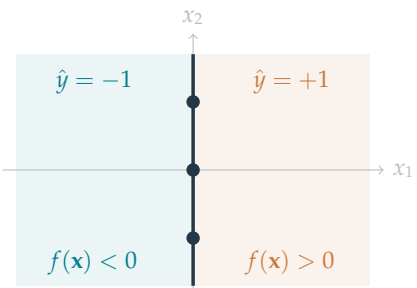
The sign changes as the score crosses zero:

$$f(\mathbf{x}) = 0 \quad \Longleftrightarrow \quad \mathbf{w}^\top \mathbf{x} + b = 0.$$

This is the decision boundary.

Example: if $f(\mathbf{x}) = x_1$, zero score means $x_1 = 0$.

Points $(0, -1)$, $(0, 0)$, $(0, 1)$, and every $(0, t)$ lie on this line.



Example: $f(\mathbf{x}) = x_1$, boundary $x_1 = 0$.

Next question How can we write all zero-score inputs as one mathematical object?

Writing the Boundary as a Set

For $f(\mathbf{x}) = x_1$, all points $(0, t)$ have zero score, for any $t \in \mathbb{R}$.

$$(0, -1), \quad (0, 0), \quad (0, 1), \quad \ldots$$

We want to describe **all** such inputs, not list them one by one.

Writing the Boundary as a Set

For $f(\mathbf{x}) = x_1$, all points $(0, t)$ have zero score, for any $t \in \mathbb{R}$.

$$(0, -1), \quad (0, 0), \quad (0, 1), \quad \ldots$$

We want to describe **all** such inputs, not list them one by one.

Name the boundary H

$$H = \{\mathbf{x} : f(\mathbf{x}) = 0\} = \{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}.$$

- H is the name of a **set of input points**.
- $\{\mathbf{x} : \text{condition}\}$ means "all inputs $\mathbf{x}$ such that the condition holds."
- Here the condition is **zero score**: substitute $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$.

Writing the Boundary as a Set

For $f(\mathbf{x}) = x_1$, all points $(0, t)$ have zero score, for any $t \in \mathbb{R}$.

$$(0, -1), \quad (0, 0), \quad (0, 1), \quad \ldots$$

We want to describe **all** such inputs, not list them one by one.

Name the boundary H

$$H = \{\mathbf{x} : f(\mathbf{x}) = 0\} = \{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}.$$

- H is the name of a **set of input points**.
- $\{\mathbf{x} : \text{condition}\}$ means "all inputs $\mathbf{x}$ such that the condition holds."
- Here the condition is **zero score**: substitute $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$.

For $\mathbf{w} \neq \mathbf{0}$, this set is a line in 2D, a plane in 3D, and a hyperplane in D dimensions.

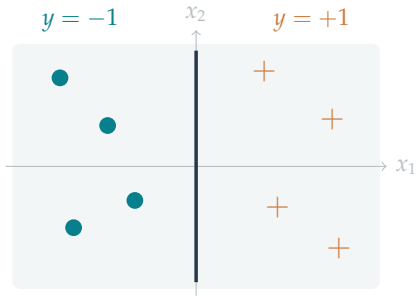
Next: does this boundary put every labeled point on the correct side?

When Does a Boundary Separate the Data?

Add the true labels to the same picture, and check which side each point lies on.

For the solid boundary $x_1 = 0$:

- Every negative example has $f(\mathbf{x}_n) < 0$.
- Every positive example has $f(\mathbf{x}_n) > 0$.



When Does a Boundary Separate the Data?

Add the true labels to the same picture, and check which side each point lies on.

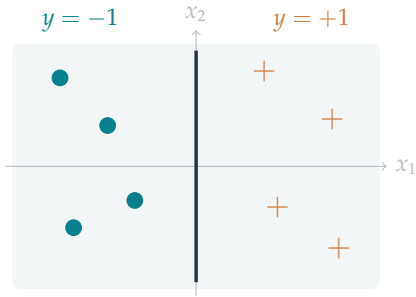
For the solid boundary $x_1 = 0$:

- Every negative example has $f(\mathbf{x}_n) < 0$.
- Every positive example has $f(\mathbf{x}_n) > 0$.

It is a separating hyperplane for these data:

$$y_n f(\mathbf{x}_n) > 0 \qquad \text{for every } n.$$

This is exactly our earlier test for correct classification, applied to **all** training points.



When Does a Boundary Separate the Data?

Add the true labels to the same picture, and check which side each point lies on.

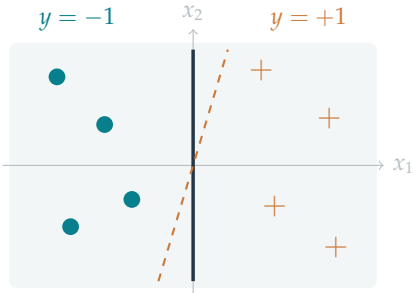
For the solid boundary $x_1 = 0$:

- Every negative example has $f(\mathbf{x}_n) < 0$.
- Every positive example has $f(\mathbf{x}_n) > 0$.

It is a separating hyperplane for these data:

$$y_n f(\mathbf{x}_n) > 0 \qquad \text{for every } n.$$

This is exactly our earlier test for correct classification, applied to **all** training points.



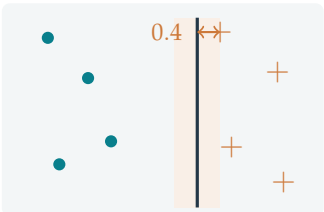
Dashed alternative: $x_1 - 0.3x_2 = 0$.

Correctness is not enough The dashed boundary separates the same points too. **Which separator should we choose?**

Correct Separation Still Leaves a Choice

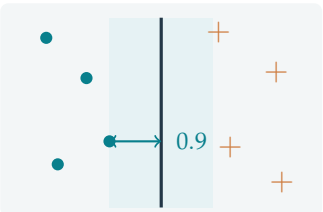
Both boundaries classify every point correctly. Compare their **nearest-point distance**.

A **Less clearance**



$H_A : \; x_1 = 0.6$

B **More clearance**

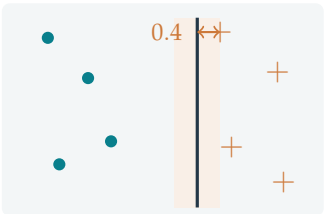


$H_B : \; x_1 = 0$

Correct Separation Still Leaves a Choice

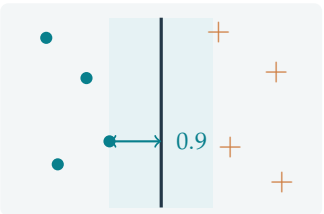
Both boundaries classify every point correctly. Compare their **nearest-point distance**.

A **Less clearance**



$H_A : \; x_1 = 0.6$

B **More clearance**



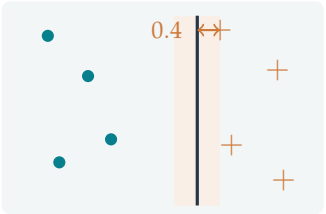
$H_B : \; x_1 = 0$

The geometric idea More clearance tolerates larger perturbations of the inputs. Choose the boundary with the **largest minimum signed distance**: the maximum margin.

Correct Separation Still Leaves a Choice

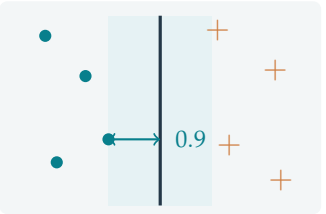
Both boundaries classify every point correctly. Compare their **nearest-point distance**.

A **Less clearance**



$H_A : \; x_1 = 0.6$

B **More clearance**



$H_B : \; x_1 = 0$

The geometric idea More clearance tolerates larger perturbations of the inputs. Choose the boundary with the **largest minimum signed distance**: the maximum margin.

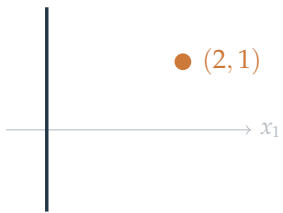
Next: turn maximum margin into a training rule for a linear classifier.

Why Can We Not Use the Score as Distance?

The same boundary can be described by differently scaled scores.

$$f_1(\mathbf{x}) = x_1, \qquad f_2(\mathbf{x}) = 3x_1.$$

Both predict $\operatorname{sign}(x_1)$ and have boundary $x_1 = 0$.



Why Can We Not Use the Score as Distance?

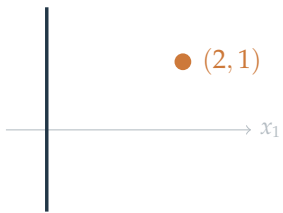
The same boundary can be described by differently scaled scores.

$$f_1(\mathbf{x}) = x_1, \qquad f_2(\mathbf{x}) = 3x_1.$$

Both predict $\operatorname{sign}(x_1)$ and have boundary $x_1 = 0$.

For $\mathbf{x} = (2, 1)^\top$ and $y = +1$:

	f_1	f_2
score $f(\mathbf{x})$	2	6
functional margin $yf(\mathbf{x})$	2	6



Why Can We Not Use the Score as Distance?

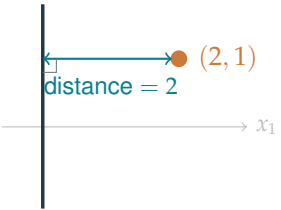
The same boundary can be described by differently scaled scores.

$$f_1(\mathbf{x}) = x_1, \qquad f_2(\mathbf{x}) = 3x_1.$$

Both predict $\operatorname{sign}(x_1)$ and have boundary $x_1 = 0$.

For $\mathbf{x} = (2, 1)^\top$ and $y = +1$:

	f_1	f_2
score $f(\mathbf{x})$	2	6
functional margin $yf(\mathbf{x})$	2	6



The contradiction The same point is distance 2 from the same boundary.

Why Can We Not Use the Score as Distance?

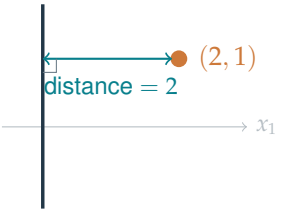
The same boundary can be described by differently scaled scores.

$$f_1(\mathbf{x}) = x_1, \qquad f_2(\mathbf{x}) = 3x_1.$$

Both predict $\operatorname{sign}(x_1)$ and have boundary $x_1 = 0$.

For $\mathbf{x} = (2, 1)^\top$ and $y = +1$:

	f_1	f_2
score $f(\mathbf{x})$	2	6
functional margin $yf(\mathbf{x})$	2	6



The contradiction The same point is distance 2 from the same boundary.

If score were distance, f_1 would report 2 but f_2 would report 6: **two different answers for the same geometric length**.

Why Can We Not Use the Score as Distance?

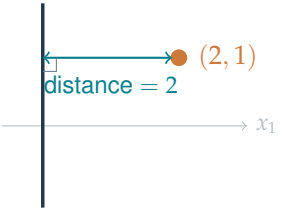
The same boundary can be described by differently scaled scores.

$$f_1(\mathbf{x}) = x_1, \qquad f_2(\mathbf{x}) = 3x_1.$$

Both predict $\operatorname{sign}(x_1)$ and have boundary $x_1 = 0$.

For $\mathbf{x} = (2, 1)^\top$ and $y = +1$:

	f_1	f_2
score $f(\mathbf{x})$	2	6
functional margin $yf(\mathbf{x})$	2	6



The contradiction The same point is distance 2 from the same boundary.

If score were distance, f_1 would report 2 but f_2 would report 6: **two different answers for the same geometric length**.

Thus neither the raw score nor $yf(\mathbf{x})$ is a geometric distance.
We need to **remove the arbitrary scale of the score**.

From Score to Distance: Find the Normal

The score has arbitrary scale. To measure distance to H, first find a perpendicular direction.

1. Test $\mathbf{w}$: pick two distinct points on H.

$$f(\mathbf{q}) = \mathbf{w}^\top \mathbf{q} + b = 0, \qquad f(\mathbf{r}) = \mathbf{w}^\top \mathbf{r} + b = 0.$$



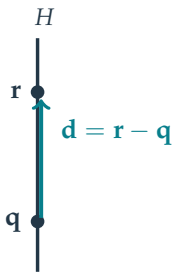
From Score to Distance: Find the Normal

The score has arbitrary scale. To measure distance to H, first find a perpendicular direction.

1. **Test $\mathbf{w}$: pick two distinct points on H.**

$$f(\mathbf{q}) = \mathbf{w}^\top \mathbf{q} + b = 0, \qquad f(\mathbf{r}) = \mathbf{w}^\top \mathbf{r} + b = 0.$$

2. **Move along H: $\mathbf{d} = \mathbf{r} - \mathbf{q}$.**



From Score to Distance: Find the Normal

The score has arbitrary scale. To measure distance to H, first find a perpendicular direction.

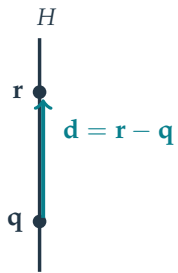
1. **Test $\mathbf{w}$: pick two distinct points on H.**

$$f(\mathbf{q}) = \mathbf{w}^\top \mathbf{q} + b = 0, \qquad f(\mathbf{r}) = \mathbf{w}^\top \mathbf{r} + b = 0.$$

2. **Move along H: $\mathbf{d} = \mathbf{r} - \mathbf{q}$.**

3. **Moving along H does not change the score.**

$$\begin{aligned} \mathbf{w}^\top \mathbf{d} &= \mathbf{w}^\top (\mathbf{r} - \mathbf{q}) \\ &= (\mathbf{w}^\top \mathbf{r} + b) - (\mathbf{w}^\top \mathbf{q} + b) = 0. \end{aligned}$$



From Score to Distance: Find the Normal

The score has arbitrary scale. To measure distance to H, first find a perpendicular direction.

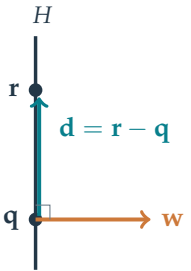
1. **Test $\mathbf{w}$: pick two distinct points on H.**

$$f(\mathbf{q}) = \mathbf{w}^\top \mathbf{q} + b = 0, \qquad f(\mathbf{r}) = \mathbf{w}^\top \mathbf{r} + b = 0.$$

2. **Move along H: $\mathbf{d} = \mathbf{r} - \mathbf{q}$.**

3. **Moving along H does not change the score.**

$$\begin{aligned} \mathbf{w}^\top \mathbf{d} &= \mathbf{w}^\top (\mathbf{r} - \mathbf{q}) \\ &= (\mathbf{w}^\top \mathbf{r} + b) - (\mathbf{w}^\top \mathbf{q} + b) = 0. \end{aligned}$$



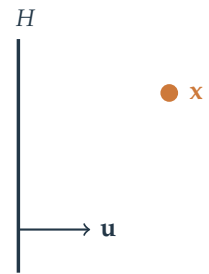
Normal vector A zero dot product means $\mathbf{w} \perp \mathbf{d}$. This holds for every direction $\mathbf{d}$ along H, so $\mathbf{w}$ is perpendicular to H.

Turn the Normal Direction into a Distance Parameter

We know the perpendicular direction. We still need a way to measure how far to move.

1. **Make a direction vector of length one.**

$$\mathbf{u} = \frac{\mathbf{w}}{\|\mathbf{w}\|}, \qquad \|\mathbf{u}\| = 1.$$

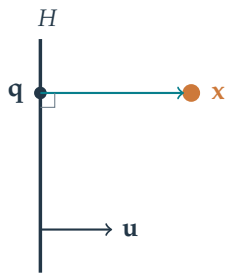


Turn the Normal Direction into a Distance Parameter

We know the perpendicular direction. We still need a way to measure how far to move.

1. Make a direction vector of length one.

$$\mathbf{u} = \frac{\mathbf{w}}{\|\mathbf{w}\|}, \qquad \|\mathbf{u}\| = 1.$$



2. From $\mathbf{x}$, follow the normal to the boundary.

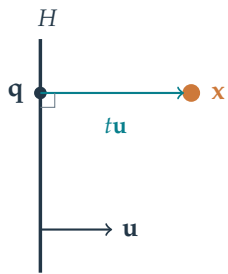
Let $\mathbf{q} \in H$ be the intersection. Then $\mathbf{x} - \mathbf{q} \parallel \mathbf{u}$.

Turn the Normal Direction into a Distance Parameter

We know the perpendicular direction. We still need a way to measure how far to move.

1. Make a direction vector of length one.

$$\mathbf{u} = \frac{\mathbf{w}}{\|\mathbf{w}\|}, \qquad \|\mathbf{u}\| = 1.$$



2. From $\mathbf{x}$, follow the normal to the boundary.

Let $\mathbf{q} \in H$ be the intersection. Then $\mathbf{x} - \mathbf{q} \parallel \mathbf{u}$.

3. Parallel vectors differ by a scalar multiplier.

$$\mathbf{x} - \mathbf{q} = t\mathbf{u} \quad \Longleftrightarrow \quad \mathbf{x} = \mathbf{q} + t\mathbf{u}.$$

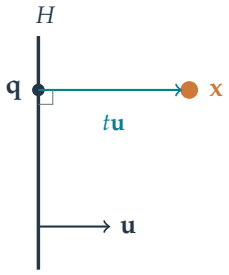
$t > 0$: along $\mathbf{u}$; $t < 0$: opposite to $\mathbf{u}$.

Turn the Normal Direction into a Distance Parameter

We know the perpendicular direction. We still need a way to measure how far to move.

1. Make a direction vector of length one.

$$\mathbf{u} = \frac{\mathbf{w}}{\|\mathbf{w}\|}, \qquad \|\mathbf{u}\| = 1.$$



2. From $\mathbf{x}$, follow the normal to the boundary.

Let $\mathbf{q} \in H$ be the intersection. Then $\mathbf{x} - \mathbf{q} \parallel \mathbf{u}$.

3. Parallel vectors differ by a scalar multiplier.

$$\mathbf{x} - \mathbf{q} = t\mathbf{u} \quad \Longleftrightarrow \quad \mathbf{x} = \mathbf{q} + t\mathbf{u}.$$

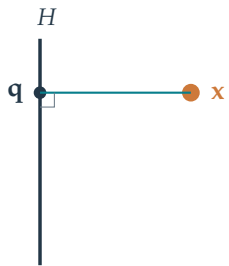
$t > 0$: along $\mathbf{u}$; $t < 0$: opposite to $\mathbf{u}$.

Why normalize? $\|\mathbf{x} - \mathbf{q}\| = \|t\mathbf{u}\| = |t|\,\|\mathbf{u}\| = |t|$. Thus $|t|$ is the segment length.

Why Is the Perpendicular the Shortest Path?

The perpendicular foot is $\mathbf{q} \in H$.

$$\mathbf{x} - \mathbf{q} = t\mathbf{u}, \qquad \|\mathbf{u}\| = 1.$$

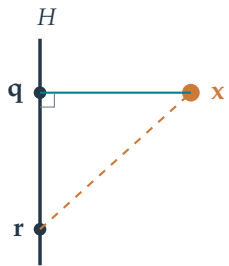


Why Is the Perpendicular the Shortest Path?

The perpendicular foot is $\mathbf{q} \in H$.

$$\mathbf{x} - \mathbf{q} = t\mathbf{u}, \qquad \|\mathbf{u}\| = 1.$$

Pick any other point $\mathbf{r}$ on the boundary. The three points form a right triangle.



Why Is the Perpendicular the Shortest Path?

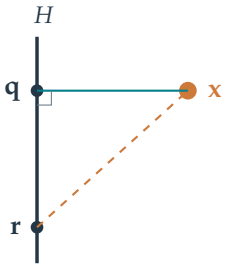
The perpendicular foot is $\mathbf{q} \in H$.

$$\mathbf{x} - \mathbf{q} = t\mathbf{u}, \qquad \|\mathbf{u}\| = 1.$$

Pick any other point $\mathbf{r}$ on the boundary. The three points form a right triangle.

$$\begin{aligned} \|\mathbf{x} - \mathbf{r}\|^2 &= \|\mathbf{x} - \mathbf{q}\|^2 + \|\mathbf{q} - \mathbf{r}\|^2 \\ &\geq \|\mathbf{x} - \mathbf{q}\|^2 . \end{aligned}$$

Every other route is at least as long as the perpendicular.



Now we know what to compute $\operatorname{dist}(\mathbf{x}, H) = \|\mathbf{x} - \mathbf{q}\| = \|t\mathbf{u}\| = |t|$. Next, recover t from the score $f(\mathbf{x})$.

Derive the Distance from the Zero-Score Condition

We have $\mathbf{x} = \mathbf{q} + t\mathbf{u}$, with $\mathbf{q} \in H$ and $\mathbf{u} = \mathbf{w}/\left\|\mathbf{w}\right\|$.

1. The foot is on the boundary, so $f(\mathbf{q}) = \mathbf{w}^\top \mathbf{q} + b = 0$.

Derive the Distance from the Zero-Score Condition

We have $\mathbf{x} = \mathbf{q} + t\mathbf{u}$, with $\mathbf{q} \in H$ and $\mathbf{u} = \mathbf{w}/\left\|\mathbf{w}\right\|$.

1. The foot is on the boundary, so $f(\mathbf{q}) = \mathbf{w}^\top \mathbf{q} + b = 0$.
2. Substitute the perpendicular decomposition into the score:

$$
\begin{aligned}
f(\mathbf{x}) &= \mathbf{w}^\top (\mathbf{q} + t\mathbf{u}) + b \\
&= \underbrace{(\mathbf{w}^\top \mathbf{q} + b)}_{=0} + \underbrace{t\, \mathbf{w}^\top \frac{\mathbf{w}}{\left\|\mathbf{w}\right\|}}_{=\left\|\mathbf{w}\right\|} = t\left\|\mathbf{w}\right\|.
\end{aligned}
$$

Derive the Distance from the Zero-Score Condition

We have $\mathbf{x} = \mathbf{q} + t\mathbf{u}$, with $\mathbf{q} \in H$ and $\mathbf{u} = \mathbf{w}/\|\mathbf{w}\|$.

1. The foot is on the boundary, so $f(\mathbf{q}) = \mathbf{w}^\top \mathbf{q} + b = 0$.
2. Substitute the perpendicular decomposition into the score:

$$\begin{aligned} f(\mathbf{x}) &= \mathbf{w}^\top(\mathbf{q} + t\mathbf{u}) + b \\ &= \underbrace{(\mathbf{w}^\top \mathbf{q} + b)}_{=0} + t\underbrace{\mathbf{w}^\top \frac{\mathbf{w}}{\|\mathbf{w}\|}}_{=\|\mathbf{w}\|} = t\|\mathbf{w}\|. \end{aligned}$$

3. Divide by $\|\mathbf{w}\|$ to recover the signed displacement t.

Distance to a hyperplane

$$t = \frac{\mathbf{w}^\top \mathbf{x} + b}{\|\mathbf{w}\|}, \qquad \operatorname{dist}(\mathbf{x}, H) = |t| = \frac{|\mathbf{w}^\top \mathbf{x} + b|}{\|\mathbf{w}\|}.$$

Derive the Distance from the Zero-Score Condition

We have $\mathbf{x} = \mathbf{q} + t\mathbf{u}$, with $\mathbf{q} \in H$ and $\mathbf{u} = \mathbf{w}/\|\mathbf{w}\|$.

1. The foot is on the boundary, so $f(\mathbf{q}) = \mathbf{w}^\top \mathbf{q} + b = 0$.
2. Substitute the perpendicular decomposition into the score:

$$\begin{aligned} f(\mathbf{x}) &= \mathbf{w}^\top(\mathbf{q} + t\mathbf{u}) + b \\ &= \underbrace{(\mathbf{w}^\top \mathbf{q} + b)}_{=0} + t\underbrace{\mathbf{w}^\top \frac{\mathbf{w}}{\|\mathbf{w}\|}}_{=\|\mathbf{w}\|} = t\|\mathbf{w}\|. \end{aligned}$$

3. Divide by $\|\mathbf{w}\|$ to recover the signed displacement t.

Distance to a hyperplane

$$t = \frac{\mathbf{w}^\top \mathbf{x} + b}{\|\mathbf{w}\|}, \qquad \operatorname{dist}(\mathbf{x}, H) = |t| = \frac{|\mathbf{w}^\top \mathbf{x} + b|}{\|\mathbf{w}\|}.$$

Check the scale: $f_1 = x_1$ gives $|2|/1 = 2$; $f_2 = 3x_1$ gives $|6|/3 = 2$.

Signed Geometric Margin: Include the True Label

The displacement $t_n = f(\mathbf{x}_n)/\|\mathbf{w}\|$ tells us the side of the boundary.
Does it also tell us whether the prediction is **correct**?

Signed Geometric Margin: Include the True Label

The displacement $t_n = f(\mathbf{x}_n)/\|\mathbf{w}\|$ tells us the side of the boundary.
Does it also tell us whether the prediction is **correct**?

Use the same label adjustment as before

$$\rho_n = y_n t_n = \frac{y_n(\mathbf{w}^\top \mathbf{x}_n + b)}{\|\mathbf{w}\|} = \frac{m_n}{\|\mathbf{w}\|}.$$

$\rho_n > 0$: correct side $\quad$ $\rho_n < 0$: wrong side $\quad$ $\rho_n = 0$: boundary.

Signed Geometric Margin: Include the True Label

The displacement $t_n = f(\mathbf{x}_n)/\|\mathbf{w}\|$ tells us the side of the boundary.
Does it also tell us whether the prediction is **correct**?

Use the same label adjustment as before

$$\rho_n = y_n t_n = \frac{y_n(\mathbf{w}^\top \mathbf{x}_n + b)}{\|\mathbf{w}\|} = \frac{m_n}{\|\mathbf{w}\|}.$$

$\rho_n > 0$: correct side $\rho_n < 0$: wrong side $\rho_n = 0$: boundary.

Example: $f(\mathbf{x}) = 3x_1$, so $\|\mathbf{w}\| = 3$.

x_1	y	$m = yf(\mathbf{x})$	$\rho = m/3$	Interpretation
2	$+1$	6	2	Correct, distance 2
-2	-1	6	2	Correct, distance 2
-2	$+1$	-6	-2	Wrong, distance 2

Signed Geometric Margin: Include the True Label

The displacement $t_n = f(\mathbf{x}_n)/\|\mathbf{w}\|$ tells us the side of the boundary.
Does it also tell us whether the prediction is **correct**?

Use the same label adjustment as before

$$\rho_n = y_n t_n = \frac{y_n(\mathbf{w}^\top \mathbf{x}_n + b)}{\|\mathbf{w}\|} = \frac{m_n}{\|\mathbf{w}\|}.$$

$\rho_n > 0$: correct side $\rho_n < 0$: wrong side $\rho_n = 0$: boundary.

Example: $f(\mathbf{x}) = 3x_1$, so $\|\mathbf{w}\| = 3$.

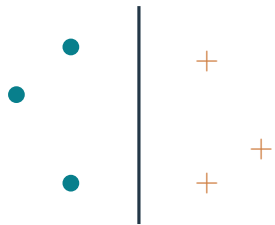
x_1	y	$m = yf(\mathbf{x})$	$\rho = m/3$	Interpretation
2	$+1$	6	2	Correct, distance 2
-2	-1	6	2	Correct, distance 2
-2	$+1$	-6	-2	Wrong, distance 2

Maximum-margin criterion: maximize the smallest ρ_n among training points.

Normalize the Score, Then Read the Margin

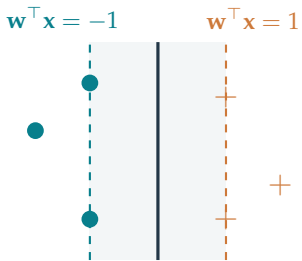
From here, set $b = 0$ to keep the optimization derivation compact.

- Decision boundary H_0: $\mathbf{w}^\top \mathbf{x} = 0$.



Normalize the Score, Then Read the Margin

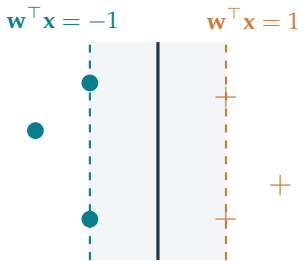
From here, set $b = 0$ to keep the optimization derivation compact.



- Decision boundary H_0: $\mathbf{w}^\top \mathbf{x} = 0$.
- Margin boundaries: $\mathbf{w}^\top \mathbf{x} = \pm 1$.

Normalize the Score, Then Read the Margin

From here, set $b = 0$ to keep the optimization derivation compact.

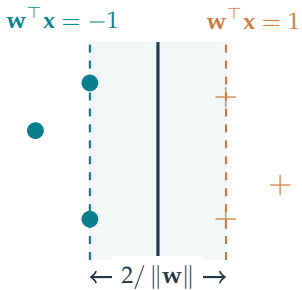


- Decision boundary H_0: $\mathbf{w}^\top \mathbf{x} = 0$.
- Margin boundaries: $\mathbf{w}^\top \mathbf{x} = \pm 1$.
- For a point $\mathbf{x}_\pm$ on either margin boundary:

$$\begin{aligned} \operatorname{dist}(\mathbf{x}_\pm, H_0) &= \frac{|\mathbf{w}^\top \mathbf{x}_\pm|}{\|\mathbf{w}\|} \\ &= \frac{|\pm 1|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}. \end{aligned}$$

Normalize the Score, Then Read the Margin

From here, set $b = 0$ to keep the optimization derivation compact.



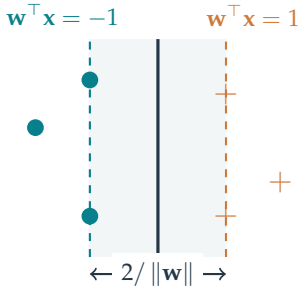
- Decision boundary H_0: $\mathbf{w}^\top \mathbf{x} = 0$.
- Margin boundaries: $\mathbf{w}^\top \mathbf{x} = \pm 1$.
- For a point $\mathbf{x}_\pm$ on either margin boundary:

$$\begin{aligned} \operatorname{dist}(\mathbf{x}_\pm, H_0) &= \frac{|\mathbf{w}^\top \mathbf{x}_\pm|}{\|\mathbf{w}\|} \\ &= \frac{|\pm 1|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}. \end{aligned}$$

- The full “street” has width $2/\|\mathbf{w}\|$.

Normalize the Score, Then Read the Margin

From here, set $b = 0$ to keep the optimization derivation compact.



- Decision boundary H_0: $\mathbf{w}^\top \mathbf{x} = 0$.
- Margin boundaries: $\mathbf{w}^\top \mathbf{x} = \pm 1$.
- For a point $\mathbf{x}_\pm$ on either margin boundary:

$$\begin{aligned} \operatorname{dist}(\mathbf{x}_\pm, H_0) &= \frac{|\mathbf{w}^\top \mathbf{x}_\pm|}{\|\mathbf{w}\|} \\ &= \frac{|\pm 1|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}. \end{aligned}$$

- The full "street" has width $2/\|\mathbf{w}\|$.

Why normalization matters With the nearest functional margin fixed at one, a smaller $\|\mathbf{w}\|$ means a larger geometric margin.

Train a Maximum-Margin Linear Classifier

Assume the data are linearly separable, and keep $b = 0$ for our derivation.

1. **Measure the nearest signed distance.**

$$\rho(\mathbf{w}) = \min_n \frac{y_n \mathbf{w}^\top \mathbf{x}_n}{\|\mathbf{w}\|}.$$

Train a Maximum-Margin Linear Classifier

Assume the data are linearly separable, and keep $b = 0$ for our derivation.

1. **Measure the nearest signed distance.**

$$\rho(\mathbf{w}) = \min_n \frac{y_n \mathbf{w}^\top \mathbf{x}_n}{\|\mathbf{w}\|}.$$

2. **Fix the arbitrary scale.** Rescale $\mathbf{w}$ so that $\min_n y_n \mathbf{w}^\top \mathbf{x}_n = 1$. Then $\rho = 1/\|\mathbf{w}\|$.

Train a Maximum-Margin Linear Classifier

Assume the data are linearly separable, and keep $b = 0$ for our derivation.

1. **Measure the nearest signed distance.**

$$\rho(\mathbf{w}) = \min_n \frac{y_n \mathbf{w}^\top \mathbf{x}_n}{\|\mathbf{w}\|}.$$

2. **Fix the arbitrary scale.** Rescale $\mathbf{w}$ so that $\min_n y_n \mathbf{w}^\top \mathbf{x}_n = 1$. Then $\rho = 1/\|\mathbf{w}\|$.

3. **Maximize the margin by minimizing the norm.**

Train a Maximum-Margin Linear Classifier

Assume the data are linearly separable, and keep $b = 0$ for our derivation.

1. **Measure the nearest signed distance.**

$$\rho(\mathbf{w}) = \min_n \frac{y_n \mathbf{w}^\top \mathbf{x}_n}{\|\mathbf{w}\|}.$$

2. **Fix the arbitrary scale.** Rescale $\mathbf{w}$ so that $\min_n y_n \mathbf{w}^\top \mathbf{x}_n = 1$. Then $\rho = 1/\|\mathbf{w}\|$.

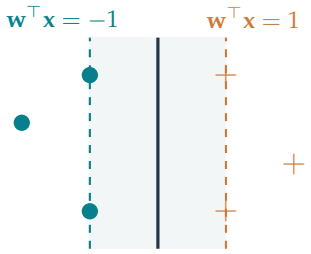
3. **Maximize the margin by minimizing the norm.**

Maximum margin with no violations

$$\min_{\mathbf{w}} \frac{1}{2}\|\mathbf{w}\|^2 \qquad \text{s.t.} \quad y_n \mathbf{w}^\top \mathbf{x}_n \geq 1 \quad \forall n.$$

Name the Classifier We Have Built

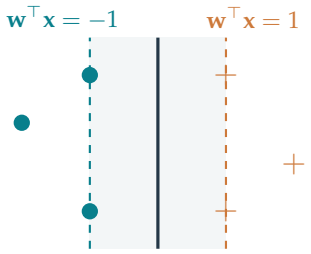
We now have a learning algorithm, not just a geometric quantity.
Training: use the labeled data to learn $\mathbf{w}$ by maximizing the margin.



Name the Classifier We Have Built

We now have a learning algorithm, not just a geometric quantity.
Training: use the labeled data to learn $\mathbf{w}$ by maximizing the margin.

$$\min_{\mathbf{w}} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_n \mathbf{w}^\top \mathbf{x}_n \geq 1 \quad \forall n.$$



Name the Classifier We Have Built

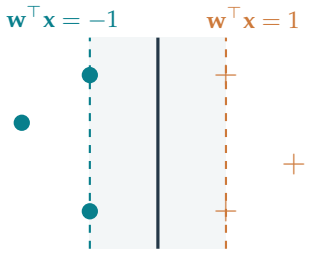
We now have a learning algorithm, not just a geometric quantity.

Training: use the labeled data to learn $\mathbf{w}$ by maximizing the margin.

$$\min_{\mathbf{w}} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_n \mathbf{w}^\top \mathbf{x}_n \geq 1 \quad \forall n.$$

Prediction: classify a new input using

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x}).$$



Name the Classifier We Have Built

We now have a learning algorithm, not just a geometric quantity.

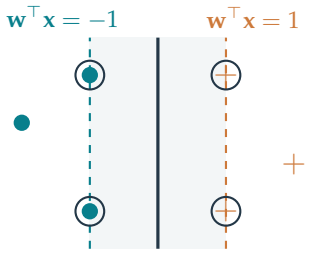
Training: use the labeled data to learn $\mathbf{w}$ by maximizing the margin.

$$\min_{\mathbf{w}} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_n \mathbf{w}^\top \mathbf{x}_n \geq 1 \quad \forall n.$$

Prediction: classify a new input using

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x}).$$

The closest training examples lie on the margin boundaries.



Circled: examples on the margin.

Name the Classifier We Have Built

We now have a learning algorithm, not just a geometric quantity.

Training: use the labeled data to learn $\mathbf{w}$ by maximizing the margin.

$$\min_{\mathbf{w}} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_n \mathbf{w}^\top \mathbf{x}_n \geq 1 \quad \forall n.$$

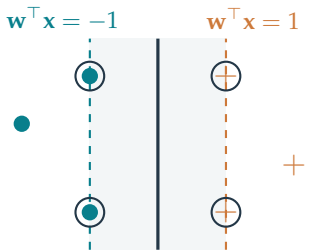
Prediction: classify a new input using

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x}).$$

The closest training examples lie on the margin boundaries.

The examples supporting the fitted boundary are called support vectors.

"Vectors" refers to their input feature vectors $\mathbf{x}_n$.



Circled: examples on the margin.

Name the Classifier We Have Built

We now have a learning algorithm, not just a geometric quantity.

Training: use the labeled data to learn $\mathbf{w}$ by maximizing the margin.

$$\min_{\mathbf{w}} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_n \mathbf{w}^\top \mathbf{x}_n \geq 1 \quad \forall n.$$

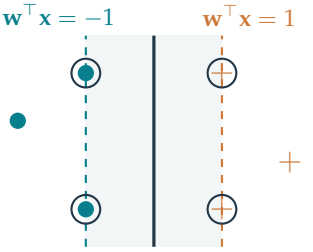
Prediction: classify a new input using

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x}).$$

The closest training examples lie on the margin boundaries.

The examples supporting the fitted boundary are called support vectors.

"Vectors" refers to their input feature vectors $\mathbf{x}_n$.



Circled: examples on the margin.

Support Vector Machine (SVM) A classifier learned using the maximum-margin principle. The training examples that support the solution give the method its name.

What Does the Margin Constraint Require?

Our optimization requires

$$y_n \mathbf{w}^\top \mathbf{x}_n \geq 1 \qquad \text{for } \textbf{every} \text{ training point } n.$$

This means **correct classification is necessary, but not sufficient**.

What Does the Margin Constraint Require?

Our optimization requires

$$y_n \mathbf{w}^\top \mathbf{x}_n \geq 1 \qquad \text{for } \textbf{every} \text{ training point } n.$$

This means **correct classification is necessary, but not sufficient**.

For a positive example ($y = +1$):

Score $\mathbf{w}^\top \mathbf{x}$	Prediction	Satisfies the margin constraint?
1.5	Correct	Yes: $1.5 \geq 1$
0.4	Correct	No: inside the margin
-0.2	Wrong	No: on the wrong side

What Does the Margin Constraint Require?

Our optimization requires

$$y_n \mathbf{w}^\top \mathbf{x}_n \geq 1 \qquad \text{for } \textbf{every} \text{ training point } n.$$

This means **correct classification is necessary, but not sufficient**.

For a positive example ($y = +1$):

Score $\mathbf{w}^\top \mathbf{x}$	Prediction	Satisfies the margin constraint?
1.5	Correct	Yes: $1.5 \geq 1$
0.4	Correct	No: inside the margin
-0.2	Wrong	No: on the wrong side

Why "hard margin"? No training point may violate the requirement $y_n \mathbf{w}^\top \mathbf{x}_n \geq 1$. The margin constraint cannot be relaxed: this is a hard-margin SVM.

What Does the Margin Constraint Require?

Our optimization requires

$$y_n \mathbf{w}^\top \mathbf{x}_n \geq 1 \qquad \text{for } \textbf{every} \text{ training point } n.$$

This means **correct classification is necessary, but not sufficient**.

For a positive example ($y = +1$):

Score $\mathbf{w}^\top \mathbf{x}$	Prediction	Satisfies the margin constraint?
1.5	Correct	Yes: $1.5 \geq 1$
0.4	Correct	No: inside the margin
-0.2	Wrong	No: on the wrong side

Why "hard margin"? No training point may violate the requirement $y_n \mathbf{w}^\top \mathbf{x}_n \geq 1$. The margin constraint cannot be relaxed: this is a hard-margin SVM.

What if no $\mathbf{w}$ satisfies all these requirements?

Relax the margin requirement, and charge a cost for violations. This leads to soft-margin SVM.

What If Perfect Separation Is Impossible?

For noisy or overlapping classes, trade margin size against the cost of violations.

What If Perfect Separation Is Impossible?

For noisy or overlapping classes, trade margin size against the cost of violations.

Soft-margin SVM

$$\min_{\mathbf{w},\boldsymbol{\xi}} \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{N}\sum_{n=1}^{N}\xi_n \quad \text{s.t.} \quad y_n\mathbf{w}^\top\mathbf{x}_n \geq 1-\xi_n, \quad \xi_n \geq 0.$$

What If Perfect Separation Is Impossible?

For noisy or overlapping classes, trade margin size against the cost of violations.

Soft-margin SVM

$$\min_{\mathbf{w},\boldsymbol{\xi}} \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{N}\sum_{n=1}^{N}\xi_n \quad \text{s.t.} \quad y_n\mathbf{w}^\top\mathbf{x}_n \geq 1-\xi_n, \quad \xi_n \geq 0.$$

We introduced N auxiliary slack variables to allow margin violations.

Can we simplify the training problem? For **any candidate classifier** $\mathbf{w}$, can we determine its best slack values directly? If so, we can eliminate $\boldsymbol{\xi}$ and optimize over $\mathbf{w}$ alone.

Which Slack Values Best Describe a Candidate Classifier?

Consider **one candidate** $\mathbf{w}$. Its scores and predictions are already determined. We compare slack assignments for this *same* classifier.

Which Slack Values Best Describe a Candidate Classifier?

Consider **one candidate** $\mathbf{w}$. Its scores and predictions are already determined. We compare slack assignments for this *same* classifier.

Example: a positive training point has score $\mathbf{w}^\top \mathbf{x}_n = 0.4$. Its constraint is $0.4 \geq 1 - \xi_n$.

Slack ξ_n	Required inequality	Feasible?	Penalty contribution
0.6	$0.4 \geq 0.4$	Yes	$0.6/N$
1.0	$0.4 \geq 0$	Yes	$1.0/N$

Which Slack Values Best Describe a Candidate Classifier?

Consider **one candidate** $\mathbf{w}$. Its scores and predictions are already determined. We compare slack assignments for this *same* classifier.

Example: a positive training point has score $\mathbf{w}^\top \mathbf{x}_n = 0.4$. Its constraint is $0.4 \geq 1 - \xi_n$.

Slack ξ_n	Required inequality	Feasible?	Penalty contribution
0.6	$0.4 \geq 0.4$	Yes	$0.6/N$
1.0	$0.4 \geq 0$	Yes	$1.0/N$

Both choices leave the classifier unchanged, but 0.6 has the smaller penalty.

Which Slack Values Best Describe a Candidate Classifier?

Consider **one candidate** $\mathbf{w}$. Its scores and predictions are already determined. We compare slack assignments for this *same* classifier.

Example: a positive training point has score $\mathbf{w}^\top \mathbf{x}_n = 0.4$. Its constraint is $0.4 \geq 1 - \xi_n$.

Slack ξ_n	Required inequality	Feasible?	Penalty contribution
0.6	$0.4 \geq 0.4$	Yes	$0.6/N$
1.0	$0.4 \geq 0$	Yes	$1.0/N$

Both choices leave the classifier unchanged, but 0.6 has the smaller penalty.

Choose the smallest feasible slack for this candidate.

Which Slack Values Best Describe a Candidate Classifier?

Consider **one candidate** $\mathbf{w}$. Its scores and predictions are already determined. We compare slack assignments for this *same* classifier.

Example: a positive training point has score $\mathbf{w}^\top \mathbf{x}_n = 0.4$. Its constraint is $0.4 \geq 1 - \xi_n$.

Slack ξ_n	Required inequality	Feasible?	Penalty contribution
0.6	$0.4 \geq 0.4$	Yes	$0.6/N$
1.0	$0.4 \geq 0$	Yes	$1.0/N$

Both choices leave the classifier unchanged, but 0.6 has the smaller penalty.

Choose the smallest feasible slack for this candidate.

In general, $\xi_n \geq 1 - y_n \mathbf{w}^\top \mathbf{x}_n$ and $\xi_n \geq 0$, so

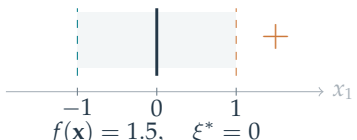
$$\xi_n^*(\mathbf{w}) = \max\{0, 1 - y_n \mathbf{w}^\top \mathbf{x}_n\}.$$

The best slack is now a **function of** $\mathbf{w}$, ready to substitute into the objective.

Read the Slack from the Point's Position

Fix $y = +1$ and $f(\mathbf{x}) = x_1$. Solid line: $f = 0$; dashed lines: $f = \pm 1$.

A. Beyond margin: correct



Read the Slack from the Point's Position

Fix $y = +1$ and $f(\mathbf{x}) = x_1$. Solid line: $f = 0$; dashed lines: $f = \pm 1$.

A. Beyond margin: correct



$f(\mathbf{x}) = 1.5, \quad \xi^* = 0$

B. Inside margin: correct



$f(\mathbf{x}) = 0.4, \quad \xi^* = 0.6$

Read the Slack from the Point's Position

Fix $y = +1$ and $f(\mathbf{x}) = x_1$. Solid line: $f = 0$; dashed lines: $f = \pm 1$.

A. Beyond margin: correct



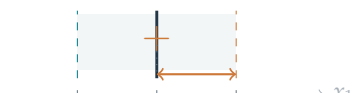
$f(\mathbf{x}) = 1.5, \quad \xi^* = 0$

B. Inside margin: correct



$f(\mathbf{x}) = 0.4, \quad \xi^* = 0.6$

C. On boundary: tie



$f(\mathbf{x}) = 0, \quad \xi^* = 1$

Read the Slack from the Point's Position

Fix $y = +1$ and $f(\mathbf{x}) = x_1$. Solid line: $f = 0$; dashed lines: $f = \pm 1$.

A. Beyond margin: correct



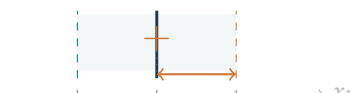
$f(\mathbf{x}) = 1.5, \quad \xi^* = 0$

B. Inside margin: correct



$f(\mathbf{x}) = 0.4, \quad \xi^* = 0.6$

C. On boundary: tie



$f(\mathbf{x}) = 0, \quad \xi^* = 1$

D. Wrong side: misclassified



$f(\mathbf{x}) = -0.5, \quad \xi^* = 1.5$

Read the Slack from the Point's Position

Fix $y = +1$ and $f(\mathbf{x}) = x_1$. Solid line: $f = 0$; dashed lines: $f = \pm 1$.

A. Beyond margin: correct



$f(\mathbf{x}) = 1.5, \quad \xi^* = 0$

B. Inside margin: correct



$f(\mathbf{x}) = 0.4, \quad \xi^* = 0.6$

C. On boundary: tie



$f(\mathbf{x}) = 0, \quad \xi^* = 1$

D. Wrong side: misclassified



$f(\mathbf{x}) = -0.5, \quad \xi^* = 1.5$

Takeaway ξ measures margin shortfall, in **score units**.
Even a correct prediction can have $\xi^* > 0$.

Motivation: The Hinge Loss (SVM)

- We just allowed margin violations using slack variables ξ_n.

Motivation: The Hinge Loss (SVM)

- We just allowed margin violations using slack variables ξ_n.
- For fixed $\mathbf{w}$, the smallest feasible slack is exactly the Hinge Loss:

$$\text{Hinge}(z, y) = [1 - yz]_+ = \max\{0, 1 - yz\}$$

Motivation: The Hinge Loss (SVM)

- We just allowed margin violations using slack variables ξ_n.
- For fixed $\mathbf{w}$, the smallest feasible slack is exactly the Hinge Loss:

$$\text{Hinge}(z, y) = [1 - yz]_+ = \max\{0, 1 - yz\}$$

- The term $\|\mathbf{w}\|^2$ controls the margin; hinge loss penalizes violations.

Motivation: The Hinge Loss (SVM)

- We just allowed margin violations using slack variables ξ_n.
- For fixed $\mathbf{w}$, the smallest feasible slack is exactly the Hinge Loss:

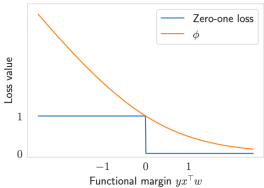
$$\text{Hinge}(z, y) = [1 - yz]_+ = \max\{0, 1 - yz\}$$

- The term $\|\mathbf{w}\|^2$ controls the margin; hinge loss penalizes violations.
- Geometry and regularized loss minimization describe the same soft-margin SVM.

The 0-1 Loss and Convex Relaxations

- The “ideal” loss for classification is the Zero-one loss:

$$L_{0-1}(\eta) = \mathbf{1}_{\eta<0} \quad \text{where } \eta = y\mathbf{x}^\top \mathbf{w}$$

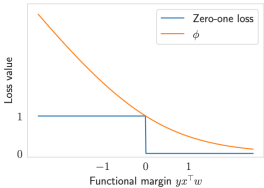


The 0-1 Loss and Convex Relaxations

- The “ideal” loss for classification is the Zero-one loss:

$$L_{0-1}(\eta) = \mathbf{1}_{\eta<0} \quad \text{where } \eta = y\mathbf{x}^\top \mathbf{w}$$

- This corresponds to Empirical Risk Minimization (ERM).



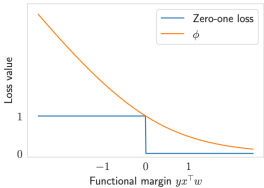
The 0-1 Loss and Convex Relaxations

- The “ideal” loss for classification is the Zero-one loss:

$$L_{0-1}(\eta) = \mathbf{1}_{\eta<0} \quad \text{where } \eta = y\mathbf{x}^\top \mathbf{w}$$

- This corresponds to Empirical Risk Minimization (ERM).

- **Problem:** The 0-1 loss is **not convex** and **not continuous**. It is computationally hard to optimize.

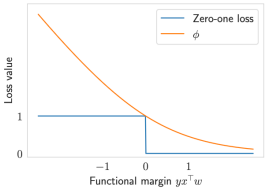


The 0-1 Loss and Convex Relaxations

- The “ideal” loss for classification is the Zero-one loss:

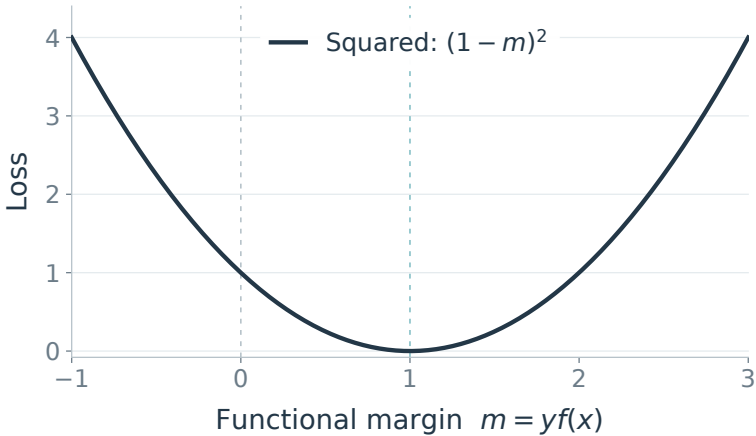
$$L_{0-1}(\eta) = \mathbf{1}_{\eta<0} \quad \text{where } \eta = y\mathbf{x}^\top \mathbf{w}$$

- This corresponds to Empirical Risk Minimization (ERM).
- **Problem:** The 0-1 loss is **not convex** and **not continuous**. It is computationally hard to optimize.
- **Solution:** Optimize a convex surrogate. Hinge loss upper-bounds the 0-1 loss; logistic loss is also convex (divide by $\log 2$ to obtain an upper bound).



Comparison of Loss Functions

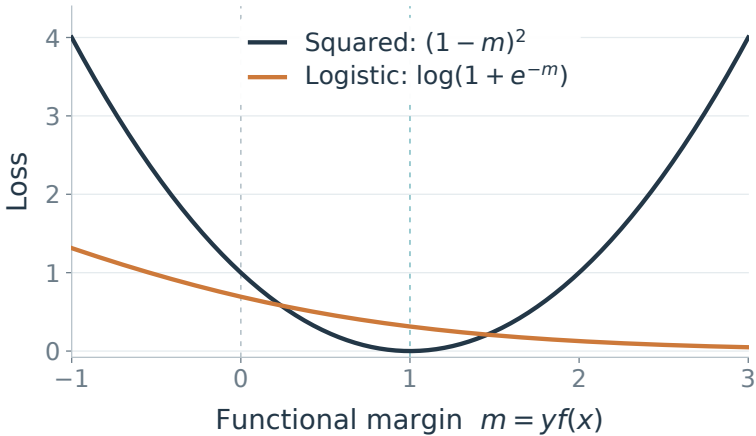
All three losses use the same functional margin $m = yf(\mathbf{x})$.



- **Squared loss**
  Targets $m = 1$; grows again for $m > 1$.

Comparison of Loss Functions

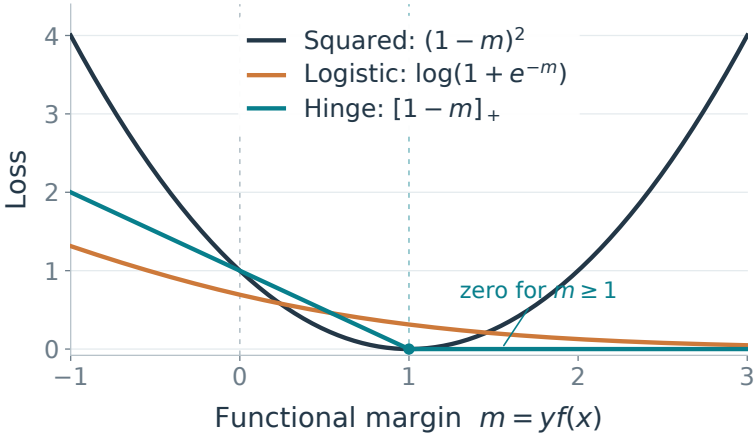
All three losses use the same functional margin $m = yf(\mathbf{x})$.



- **Squared loss**
  Targets $m = 1$; grows again for $m > 1$.
- **Logistic loss**
  Decreases smoothly; stays positive for every finite m.

Comparison of Loss Functions

All three losses use the same functional margin $m = yf(\mathbf{x})$.



- **Squared loss**
  Targets $m = 1$; grows again for $m > 1$.
- **Logistic loss**
  Decreases smoothly; stays positive for every finite m.
- **Hinge loss**
  Zero for $m \geq 1$; only penalizes margin violations.

Support Vector Machine Optimization Problem

SVMs solve the following optimization problem:

$$\min_{\mathbf{w}} \underbrace{\frac{1}{N}\sum_{n=1}^{N}\left[1 - y_n \mathbf{x}_n^\top \mathbf{w}\right]_+}_{\text{Hinge Loss (Empirical Risk)}} + \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|^2}_{\text{Regularization}}$$

Support Vector Machine Optimization Problem

SVMs solve the following optimization problem:

$$\min_{\mathbf{w}} \underbrace{\frac{1}{N}\sum_{n=1}^{N}\left[1 - y_n \mathbf{x}_n^\top \mathbf{w}\right]_+}_{\text{Hinge Loss (Empirical Risk)}} + \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|^2}_{\text{Regularization}}$$

- The **Hinge Loss** pushes for correct classification, penalizing points in the margin ($yz < 1$).
- The **Regularization** ($\|\mathbf{w}\|^2$) pushes for a simpler model.

Support Vector Machine Optimization Problem

SVMs solve the following optimization problem:

$$\min_{\mathbf{w}} \underbrace{\frac{1}{N}\sum_{n=1}^{N}\left[1 - y_n \mathbf{x}_n^\top \mathbf{w}\right]_+}_{\text{Hinge Loss (Empirical Risk)}} + \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|^2}_{\text{Regularization}}$$

- The **Hinge Loss** pushes for correct classification, penalizing points in the margin ($yz < 1$).
- The **Regularization** ($\|\mathbf{w}\|^2$) pushes for a simpler model.
- **Geometric Meaning:** Minimizing $\|\mathbf{w}\|^2$ is equivalent to **maximizing the margin**.

Support Vector Machine Optimization Problem

SVMs solve the following optimization problem:

$$\min_{\mathbf{w}} \underbrace{\frac{1}{N}\sum_{n=1}^{N}\left[1 - y_n \mathbf{x}_n^\top \mathbf{w}\right]_+}_{\text{Hinge Loss (Empirical Risk)}} + \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|^2}_{\text{Regularization}}$$

- The **Hinge Loss** pushes for correct classification, penalizing points in the margin ($yz < 1$).
- The **Regularization** ($\|\mathbf{w}\|^2$) pushes for a simpler model.
- **Geometric Meaning:** Minimizing $\|\mathbf{w}\|^2$ is equivalent to **maximizing the margin**.
- λ is a hyperparameter that controls the trade-off:
  - **Small λ:** Weaker regularization; more emphasis on reducing hinge loss.
  - **Large λ:** Stronger regularization; smaller weights, allowing more margin violations.

Optimization (Primal)

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} \left[1 - y_n \mathbf{x}_n^\top \mathbf{w}\right]_+ + \tfrac{\lambda}{2} \|\mathbf{w}\|^2$$

- Is this function convex? (Sum of convex functions: Hinge loss is convex, $\|\mathbf{w}\|^2$ is convex).

Optimization (Primal)

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} \left[1 - y_n \mathbf{x}_n^\top \mathbf{w}\right]_+ + \tfrac{\lambda}{2} \|\mathbf{w}\|^2$$

- Is this function convex? **Yes.** (Sum of convex functions: Hinge loss is convex, $\|\mathbf{w}\|^2$ is convex).

Optimization (Primal)

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} \left[1 - y_n \mathbf{x}_n^\top \mathbf{w}\right]_+ + \tfrac{\lambda}{2} \|\mathbf{w}\|^2$$

- Is this function convex? **Yes.** (Sum of convex functions: Hinge loss is convex, $\|\mathbf{w}\|^2$ is convex).
- Is it differentiable? (The Hinge loss has a "kink" at $1 - y_n \mathbf{x}_n^\top \mathbf{w} = 0$).

Optimization (Primal)

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} \left[1 - y_n \mathbf{x}_n^\top \mathbf{w}\right]_+ + \tfrac{\lambda}{2} \|\mathbf{w}\|^2$$

- Is this function convex? **Yes.** (Sum of convex functions: Hinge loss is convex, $\|\mathbf{w}\|^2$ is convex).
- Is it differentiable? **No.** (The Hinge loss has a "kink" at $1 - y_n \mathbf{x}_n^\top \mathbf{w} = 0$).

Optimization (Primal)

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} \left[1 - y_n \mathbf{x}_n^\top \mathbf{w}\right]_+ + \tfrac{\lambda}{2} \|\mathbf{w}\|^2$$

- Is this function convex? **Yes.** (Sum of convex functions: Hinge loss is convex, $\|\mathbf{w}\|^2$ is convex).
- Is it differentiable? **No.** (The Hinge loss has a "kink" at $1 - y_n \mathbf{x}_n^\top \mathbf{w} = 0$).
- Because it is convex, we can optimize it using subgradient methods, like Stochastic Subgradient Descent (SGD).

Optimization (Primal)

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} \left[1 - y_n \mathbf{x}_n^\top \mathbf{w}\right]_+ + \tfrac{\lambda}{2} \|\mathbf{w}\|^2$$

- Is this function convex? **Yes.** (Sum of convex functions: Hinge loss is convex, $\|\mathbf{w}\|^2$ is convex).
- Is it differentiable? **No.** (The Hinge loss has a "kink" at $1 - y_n \mathbf{x}_n^\top \mathbf{w} = 0$).
- Because it is convex, we can optimize it using subgradient methods, like Stochastic Subgradient Descent (SGD).
- Another formulation will expose a quadratic problem and, later, inner products between examples. We reach it through duality.

Why Can Hinge Loss Be Written as a Maximum?

Write the hinge loss as $[z]_+ = \max(0, z)$, where $z = 1 - yf(\mathbf{x})$.

Why Can Hinge Loss Be Written as a Maximum?

Write the hinge loss as $[z]_+ = \max(0, z)$, where $z = 1 - yf(\mathbf{x})$.
Consider a number $\alpha \in [0, 1]$. How large can αz be?

Value of z	Best choice of α	Maximum αz	Hinge loss
$z > 0$	$\alpha = 1$	z	z
$z < 0$	$\alpha = 0$	0	0
$z = 0$	Any $\alpha \in [0, 1]$	0	0

Why Can Hinge Loss Be Written as a Maximum?

Write the hinge loss as $[z]_+ = \max(0, z)$, where $z = 1 - yf(\mathbf{x})$.
Consider a number $\alpha \in [0, 1]$. How large can αz be?

Value of z	Best choice of α	Maximum αz	Hinge loss
$z > 0$	$\alpha = 1$	z	z
$z < 0$	$\alpha = 0$	0	0
$z = 0$	Any $\alpha \in [0, 1]$	0	0

An exact identity

$$[z]_+ = \max_{0 \le \alpha \le 1} \alpha z.$$

The right side gives the same value for **every** z.

Why Can Hinge Loss Be Written as a Maximum?

Write the hinge loss as $[z]_+ = \max(0, z)$, where $z = 1 - yf(\mathbf{x})$.
Consider a number $\alpha \in [0, 1]$. How large can αz be?

Value of z	Best choice of α	Maximum αz	Hinge loss
$z > 0$	$\alpha = 1$	z	z
$z < 0$	$\alpha = 0$	0	0
$z = 0$	Any $\alpha \in [0, 1]$	0	0

An exact identity

$$[z]_+ = \max_{0 \le \alpha \le 1} \alpha z.$$

The right side gives the same value for **every** z.

Check: $z = 0.6$ gives $\max_\alpha 0.6\alpha = 0.6$; $z = -0.5$ gives $\max_\alpha(-0.5\alpha) = 0$.

One Auxiliary Variable for Each Training Example

For a **given classifier** $\mathbf{w}$, let $z_n = 1 - y_n \mathbf{x}_n^\top \mathbf{w}$. Apply the identity to each hinge term:

$$[z_n]_+ = \max_{\alpha_n \in [0,1]} \alpha_n z_n.$$

One Auxiliary Variable for Each Training Example

For a **given classifier** $\mathbf{w}$, let $z_n = 1 - y_n \mathbf{x}_n^\top \mathbf{w}$. Apply the identity to each hinge term:

$$[z_n]_+ = \max_{\alpha_n \in [0,1]} \alpha_n z_n.$$

Each α_n can be chosen independently, so

$$\frac{1}{N} \sum_{n=1}^{N} [z_n]_+ = \max_{\boldsymbol{\alpha} \in [0,1]^N} \frac{1}{N} \sum_{n=1}^{N} \alpha_n z_n.$$

One Auxiliary Variable for Each Training Example

For a **given classifier** $\mathbf{w}$, let $z_n = 1 - y_n \mathbf{x}_n^\top \mathbf{w}$. Apply the identity to each hinge term:

$$[z_n]_+ = \max_{\alpha_n \in [0,1]} \alpha_n z_n.$$

Each α_n can be chosen independently, so

$$\frac{1}{N} \sum_{n=1}^{N} [z_n]_+ = \max_{\boldsymbol{\alpha} \in [0,1]^N} \frac{1}{N} \sum_{n=1}^{N} \alpha_n z_n.$$

Two-point example: $z_1 = 0.6$, $z_2 = -0.5$.

$$\max_{\alpha_1, \alpha_2 \in [0,1]} \frac{1}{2}(0.6\alpha_1 - 0.5\alpha_2) = \frac{1}{2}(0.6) = 0.3.$$

Choose $\alpha_1 = 1$, $\alpha_2 = 0$: exactly the average hinge loss.

One Auxiliary Variable for Each Training Example

For a **given classifier** $\mathbf{w}$, let $z_n = 1 - y_n \mathbf{x}_n^\top \mathbf{w}$. Apply the identity to each hinge term:

$$[z_n]_+ = \max_{\alpha_n \in [0,1]} \alpha_n z_n.$$

Each α_n can be chosen independently, so

$$\frac{1}{N}\sum_{n=1}^{N} [z_n]_+ = \max_{\boldsymbol{\alpha} \in [0,1]^N} \frac{1}{N}\sum_{n=1}^{N} \alpha_n z_n.$$

Two-point example: $z_1 = 0.6$, $z_2 = -0.5$.

$$\max_{\alpha_1, \alpha_2 \in [0,1]} \frac{1}{2}(0.6\alpha_1 - 0.5\alpha_2) = \frac{1}{2}(0.6) = 0.3.$$

Choose $\alpha_1 = 1$, $\alpha_2 = 0$: exactly the average hinge loss.

What does α_n do here?

It recovers the positive part of the margin violation. The examples and labels stay fixed; these are **auxiliary optimization variables**.

The SVM Is Now a Min-Max Problem

The primal objective is

$$\mathcal{L}(\mathbf{w}) = \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{N} \sum_n [1 - y_n \mathbf{x}_n^\top \mathbf{w}]_+.$$

The SVM Is Now a Min-Max Problem

The primal objective is

$$\mathcal{L}(\mathbf{w}) = \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{N} \sum_n [1 - y_n \mathbf{x}_n^\top \mathbf{w}]_+.$$

The norm term does not depend on $\boldsymbol{\alpha}$.

The SVM Is Now a Min-Max Problem

The primal objective is

$$\mathcal{L}(\mathbf{w}) = \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{N} \sum_n [1 - y_n \mathbf{x}_n^\top \mathbf{w}]_+.$$

The norm term does not depend on $\boldsymbol{\alpha}$.
The hinge identity gives

$$\mathcal{L}(\mathbf{w}) = \max_{\boldsymbol{\alpha} \in [0,1]^N} G(\mathbf{w}, \boldsymbol{\alpha}), \tag{1}$$

$$G(\mathbf{w}, \boldsymbol{\alpha}) = \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{N} \sum_n \alpha_n (1 - y_n \mathbf{x}_n^\top \mathbf{w}). \tag{2}$$

The SVM Is Now a Min-Max Problem

The primal objective is

$$\mathcal{L}(\mathbf{w}) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{N}\sum_n [1 - y_n \mathbf{x}_n^\top \mathbf{w}]_+.$$

The norm term does not depend on $\boldsymbol{\alpha}$.
The hinge identity gives

$$\mathcal{L}(\mathbf{w}) = \max_{\boldsymbol{\alpha}\in[0,1]^N} G(\mathbf{w}, \boldsymbol{\alpha}), \tag{1}$$

$$G(\mathbf{w}, \boldsymbol{\alpha}) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{N}\sum_n \alpha_n (1 - y_n \mathbf{x}_n^\top \mathbf{w}). \tag{2}$$

Read from the inside out For each candidate $\mathbf{w}$, the inner maximum computes its **exact SVM loss**. The outer minimum then chooses the classifier with the smallest loss:

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \min_{\mathbf{w}} \max_{\boldsymbol{\alpha}\in[0,1]^N} G(\mathbf{w}, \boldsymbol{\alpha}).$$

Why Look for a Dual Form?

The min-max representation exposes a useful structure. For **fixed** $\boldsymbol{\alpha}$,

$$G(\mathbf{w}, \boldsymbol{\alpha}) = \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|^2}_{\text{quadratic in } \mathbf{w}} \underbrace{- \frac{1}{N}\sum_n \alpha_n y_n \mathbf{x}_n^\top \mathbf{w}}_{\text{linear in } \mathbf{w}} + \underbrace{\frac{1}{N}\sum_n \alpha_n}_{\text{constant in } \mathbf{w}} .$$

Why Look for a Dual Form?

The min-max representation exposes a useful structure. For **fixed** $\boldsymbol{\alpha}$,

$$G(\mathbf{w}, \boldsymbol{\alpha}) = \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|^2}_{\text{quadratic in } \mathbf{w}} \underbrace{- \frac{1}{N}\sum_n \alpha_n y_n \mathbf{x}_n^\top \mathbf{w}}_{\text{linear in } \mathbf{w}} + \underbrace{\frac{1}{N}\sum_n \alpha_n}_{\text{constant in } \mathbf{w}} .$$

1. Eliminate the classifier parameters analytically.

This is a smooth quadratic in $\mathbf{w}$. We can find its minimum by setting $\nabla_{\mathbf{w}} G = 0$, giving $\mathbf{w}(\boldsymbol{\alpha})$.

Why Look for a Dual Form?

The min-max representation exposes a useful structure. For **fixed** $\boldsymbol{\alpha}$,

$$G(\mathbf{w}, \boldsymbol{\alpha}) = \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|^2}_{\text{quadratic in } \mathbf{w}} \underbrace{- \frac{1}{N}\sum_n \alpha_n y_n \mathbf{x}_n^\top \mathbf{w}}_{\text{linear in } \mathbf{w}} + \underbrace{\frac{1}{N}\sum_n \alpha_n}_{\text{constant in } \mathbf{w}} .$$

1. Eliminate the classifier parameters analytically.

This is a smooth quadratic in $\mathbf{w}$. We can find its minimum by setting $\nabla_{\mathbf{w}} G = 0$, giving $\mathbf{w}(\boldsymbol{\alpha})$.

2. Obtain a problem in sample weights.

Substitution leaves $g(\boldsymbol{\alpha})$ with N example weights α_n, instead of D feature coefficients.

Why Look for a Dual Form?

The min-max representation exposes a useful structure. For **fixed** $\boldsymbol{\alpha}$,

$$G(\mathbf{w}, \boldsymbol{\alpha}) = \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|^2}_{\text{quadratic in } \mathbf{w}} \underbrace{- \frac{1}{N}\sum_n \alpha_n y_n \mathbf{x}_n^\top \mathbf{w}}_{\text{linear in } \mathbf{w}} + \underbrace{\frac{1}{N}\sum_n \alpha_n}_{\text{constant in } \mathbf{w}} .$$

1. Eliminate the classifier parameters analytically.

This is a smooth quadratic in $\mathbf{w}$. We can find its minimum by setting $\nabla_{\mathbf{w}} G = 0$, giving $\mathbf{w}(\boldsymbol{\alpha})$.

2. Obtain a problem in sample weights.

Substitution leaves $g(\boldsymbol{\alpha})$ with N example weights α_n, instead of D feature coefficients.

3. Reveal structure that the primal hides.

The dual will reveal support-vector weights and depend on inner products $\mathbf{x}_i^\top \mathbf{x}_j$, enabling kernels.

Why Look for a Dual Form?

The min-max representation exposes a useful structure. For **fixed** $\boldsymbol{\alpha}$,

$$G(\mathbf{w}, \boldsymbol{\alpha}) = \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|^2}_{\text{quadratic in } \mathbf{w}} \underbrace{- \frac{1}{N}\sum_n \alpha_n y_n \mathbf{x}_n^\top \mathbf{w}}_{\text{linear in } \mathbf{w}} + \underbrace{\frac{1}{N}\sum_n \alpha_n}_{\text{constant in } \mathbf{w}} .$$

1. Eliminate the classifier parameters analytically.

This is a smooth quadratic in $\mathbf{w}$. We can find its minimum by setting $\nabla_{\mathbf{w}} G = 0$, giving $\mathbf{w}(\boldsymbol{\alpha})$.

2. Obtain a problem in sample weights.

Substitution leaves $g(\boldsymbol{\alpha})$ with N example weights α_n, instead of D feature coefficients.

3. Reveal structure that the primal hides.

The dual will reveal support-vector weights and depend on inner products $\mathbf{x}_i^\top \mathbf{x}_j$, enabling kernels.

Next step Eliminating $\mathbf{w}$ first requires reversing the min-max order. We must check that the optimal value is preserved.

The Dual Changes the Order of Optimization

Primal: for each candidate $\mathbf{w}$, first find its maximizing $\boldsymbol{\alpha}$; then choose the best $\mathbf{w}$.

$$p^* = \min_{\mathbf{w}} \underbrace{\max_{\boldsymbol{\alpha} \in [0,1]^N} G(\mathbf{w}, \boldsymbol{\alpha})}_{= \mathcal{L}(\mathbf{w})} .$$

The Dual Changes the Order of Optimization

Primal: for each candidate $\mathbf{w}$, first find its maximizing $\boldsymbol{\alpha}$; then choose the best $\mathbf{w}$.

$$p^* = \min_{\mathbf{w}} \underbrace{\max_{\boldsymbol{\alpha} \in [0,1]^N} G(\mathbf{w}, \boldsymbol{\alpha})}_{=\mathcal{L}(\mathbf{w})}.$$

Dual: for each candidate $\boldsymbol{\alpha}$, first find its minimizing $\mathbf{w}$.

$$g(\boldsymbol{\alpha}) = \min_{\mathbf{w}} G(\mathbf{w}, \boldsymbol{\alpha}).$$

The Dual Changes the Order of Optimization

Primal: for each candidate $\mathbf{w}$, first find its maximizing $\boldsymbol{\alpha}$; then choose the best $\mathbf{w}$.

$$p^* = \min_{\mathbf{w}} \underbrace{\max_{\boldsymbol{\alpha} \in [0,1]^N} G(\mathbf{w}, \boldsymbol{\alpha})}_{=\mathcal{L}(\mathbf{w})}.$$

Dual: for each candidate $\boldsymbol{\alpha}$, first find its minimizing $\mathbf{w}$.

$$g(\boldsymbol{\alpha}) = \min_{\mathbf{w}} G(\mathbf{w}, \boldsymbol{\alpha}).$$

Then choose the $\boldsymbol{\alpha}$ giving the largest value:

$$d^* = \max_{\boldsymbol{\alpha} \in [0,1]^N} \underbrace{\min_{\mathbf{w}} G(\mathbf{w}, \boldsymbol{\alpha})}_{=g(\boldsymbol{\alpha})}.$$

The question Both use the same G, but the inner optimization is different. Does changing the order change the final answer?

A Four-Number Example: Order Can Matter

A toy G has two possible rows $\mathbf{w}$ and two possible columns $\boldsymbol{\alpha}$.

$G(\mathbf{w}, \boldsymbol{\alpha})$	$\boldsymbol{\alpha}_A$	$\boldsymbol{\alpha}_B$	
$\mathbf{w}_A$	0	1	
$\mathbf{w}_B$	1	0	

A Four-Number Example: Order Can Matter

A toy G has two possible rows $\mathbf{w}$ and two possible columns $\boldsymbol{\alpha}$.

$G(\mathbf{w}, \boldsymbol{\alpha})$	$\boldsymbol{\alpha}_A$	$\boldsymbol{\alpha}_B$	Row maximum
$\mathbf{w}_A$	0	1	1
$\mathbf{w}_B$	1	0	1

Max within each row, then min over rows:

$$\min_{\mathbf{w}} \max_{\boldsymbol{\alpha}} G(\mathbf{w}, \boldsymbol{\alpha}) = \min\{1, 1\} = 1.$$

A Four-Number Example: Order Can Matter

A toy G has two possible rows $\mathbf{w}$ and two possible columns $\boldsymbol{\alpha}$.

$G(\mathbf{w}, \boldsymbol{\alpha})$	$\boldsymbol{\alpha}_A$	$\boldsymbol{\alpha}_B$	Row maximum
$\mathbf{w}_A$	0	1	1
$\mathbf{w}_B$	1	0	1
Column minimum	0	0	

Max within each row, then min over rows:

$$\min_{\mathbf{w}} \max_{\boldsymbol{\alpha}} G(\mathbf{w}, \boldsymbol{\alpha}) = \min\{1, 1\} = 1.$$

Min within each column, then max over columns:

$$\max_{\boldsymbol{\alpha}} \min_{\mathbf{w}} G(\mathbf{w}, \boldsymbol{\alpha}) = \max\{0, 0\} = 0.$$

A Four-Number Example: Order Can Matter

A toy G has two possible rows $\mathbf{w}$ and two possible columns $\boldsymbol{\alpha}$.

$G(\mathbf{w}, \boldsymbol{\alpha})$	$\boldsymbol{\alpha}_A$	$\boldsymbol{\alpha}_B$	Row maximum
$\mathbf{w}_A$	0	1	1
$\mathbf{w}_B$	1	0	1
Column minimum	0	0	

Max within each row, then min over rows:

$$\min_{\mathbf{w}} \max_{\boldsymbol{\alpha}} G(\mathbf{w}, \boldsymbol{\alpha}) = \min\{1, 1\} = 1.$$

Min within each column, then max over columns:

$$\max_{\boldsymbol{\alpha}} \min_{\mathbf{w}} G(\mathbf{w}, \boldsymbol{\alpha}) = \max\{0, 0\} = 0.$$

Different answers Same four entries, but the two orders give 1 and 0. Equality for our SVM needs an additional argument.

What Can We Say Before Proving Equality?

For any fixed pair $(\mathbf{w}, \boldsymbol{\alpha})$, a minimum is no larger than that entry, and a maximum is no smaller:

$$\underbrace{\min_{\mathbf{v}} G(\mathbf{v}, \boldsymbol{\alpha})}_{g(\boldsymbol{\alpha})} \;\leq\; G(\mathbf{w}, \boldsymbol{\alpha}) \;\leq\; \underbrace{\max_{\boldsymbol{\beta} \in [0,1]^N} G(\mathbf{w}, \boldsymbol{\beta})}_{\mathcal{L}(\mathbf{w})}.$$

What Can We Say Before Proving Equality?

For any fixed pair $(\mathbf{w}, \boldsymbol{\alpha})$, a minimum is no larger than that entry, and a maximum is no smaller:

$$\underbrace{\min_{\mathbf{v}} G(\mathbf{v}, \boldsymbol{\alpha})}_{g(\boldsymbol{\alpha})} \;\leq\; G(\mathbf{w}, \boldsymbol{\alpha}) \;\leq\; \underbrace{\max_{\boldsymbol{\beta} \in [0,1]^N} G(\mathbf{w}, \boldsymbol{\beta})}_{\mathcal{L}(\mathbf{w})}.$$

Thus **every** dual candidate value is below **every** primal candidate value:

$$g(\boldsymbol{\alpha}) \leq \mathcal{L}(\mathbf{w}) \qquad \text{for all } \boldsymbol{\alpha}, \mathbf{w}.$$

What Can We Say Before Proving Equality?

For any fixed pair $(\mathbf{w}, \boldsymbol{\alpha})$, a minimum is no larger than that entry, and a maximum is no smaller:

$$\underbrace{\min_{\mathbf{v}} G(\mathbf{v}, \boldsymbol{\alpha})}_{g(\boldsymbol{\alpha})} \;\leq\; G(\mathbf{w}, \boldsymbol{\alpha}) \;\leq\; \underbrace{\max_{\boldsymbol{\beta} \in [0,1]^N} G(\mathbf{w}, \boldsymbol{\beta})}_{\mathcal{L}(\mathbf{w})}.$$

Thus **every** dual candidate value is below **every** primal candidate value:

$$g(\boldsymbol{\alpha}) \leq \mathcal{L}(\mathbf{w}) \qquad \text{for all } \boldsymbol{\alpha}, \mathbf{w}.$$

Even the largest dual value cannot exceed the smallest primal value:

$$\underbrace{\max_{\boldsymbol{\alpha}} g(\boldsymbol{\alpha})}_{d^*} \leq \underbrace{\min_{\mathbf{w}} \mathcal{L}(\mathbf{w})}_{p^*}.$$

This is weak duality: the dual gives a **lower bound**.

What Can We Say Before Proving Equality?

For any fixed pair $(\mathbf{w}, \boldsymbol{\alpha})$, a minimum is no larger than that entry, and a maximum is no smaller:

$$\underbrace{\min_{\mathbf{v}} G(\mathbf{v}, \boldsymbol{\alpha})}_{g(\boldsymbol{\alpha})} \;\le\; G(\mathbf{w}, \boldsymbol{\alpha}) \;\le\; \underbrace{\max_{\boldsymbol{\beta} \in [0,1]^N} G(\mathbf{w}, \boldsymbol{\beta})}_{\mathcal{L}(\mathbf{w})}.$$

Thus **every** dual candidate value is below **every** primal candidate value:

$$g(\boldsymbol{\alpha}) \le \mathcal{L}(\mathbf{w}) \qquad \text{for all } \boldsymbol{\alpha}, \mathbf{w}.$$

Even the largest dual value cannot exceed the smallest primal value:

$$\underbrace{\max_{\boldsymbol{\alpha}} g(\boldsymbol{\alpha})}_{d^*} \le \underbrace{\min_{\mathbf{w}} \mathcal{L}(\mathbf{w})}_{p^*}.$$

This is weak duality: the dual gives a **lower bound**.

Strong duality If $d^* = p^*$, the best dual bound reaches the primal optimum. Next we check why this equality holds for soft-margin SVM.

Why Can We Use the Dual for This SVM?

For $\lambda > 0$, the soft-margin primal is a convex problem with affine constraints:

$$\min_{\mathbf{w},\boldsymbol{\xi}} \ \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{N}\sum_n \xi_n, \qquad y_n \mathbf{x}_n^\top \mathbf{w} \geq 1 - \xi_n, \quad \xi_n \geq 0.$$

Why Can We Use the Dual for This SVM?

For $\lambda > 0$, the soft-margin primal is a convex problem with affine constraints:

$$\min_{\mathbf{w},\boldsymbol{\xi}} \ \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{N}\sum_n \xi_n, \qquad y_n \mathbf{x}_n^\top \mathbf{w} \geq 1 - \xi_n, \quad \xi_n \geq 0.$$

A strictly feasible choice always exists: take $\mathbf{w} = \mathbf{0}$ and every $\xi_n = 2$. Then $0 > 1 - 2 = -1$ and $2 > 0$, so every inequality holds strictly.

Why Can We Use the Dual for This SVM?

For $\lambda > 0$, the soft-margin primal is a convex problem with affine constraints:

$$\min_{\mathbf{w},\boldsymbol{\xi}} \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{N}\sum_n \xi_n, \qquad y_n \mathbf{x}_n^\top \mathbf{w} \geq 1 - \xi_n, \quad \xi_n \geq 0.$$

A strictly feasible choice always exists: take $\mathbf{w} = \mathbf{0}$ and every $\xi_n = 2$. Then $0 > 1 - 2 = -1$ and $2 > 0$, so every inequality holds strictly.

Strong duality for soft-margin SVM This is Slater's condition for a convex problem. It guarantees

$$\min_{\mathbf{w}} \max_{\boldsymbol{\alpha}\in[0,1]^N} G(\mathbf{w},\boldsymbol{\alpha}) = \max_{\boldsymbol{\alpha}\in[0,1]^N} \min_{\mathbf{w}} G(\mathbf{w},\boldsymbol{\alpha}).$$

Why Can We Use the Dual for This SVM?

For $\lambda > 0$, the soft-margin primal is a convex problem with affine constraints:

$$\min_{\mathbf{w},\boldsymbol{\xi}} \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{N}\sum_n \xi_n, \qquad y_n \mathbf{x}_n^\top \mathbf{w} \geq 1 - \xi_n, \quad \xi_n \geq 0.$$

A strictly feasible choice always exists: take $\mathbf{w} = \mathbf{0}$ and every $\xi_n = 2$. Then $0 > 1 - 2 = -1$ and $2 > 0$, so every inequality holds strictly.

Strong duality for soft-margin SVM This is Slater's condition for a convex problem. It guarantees

$$\min_{\mathbf{w}} \max_{\boldsymbol{\alpha}\in[0,1]^N} G(\mathbf{w},\boldsymbol{\alpha}) = \max_{\boldsymbol{\alpha}\in[0,1]^N} \min_{\mathbf{w}} G(\mathbf{w},\boldsymbol{\alpha}).$$

Why useful? For fixed $\boldsymbol{\alpha}$, G is a quadratic in $\mathbf{w}$. We can minimize it analytically and obtain an optimization problem over $\boldsymbol{\alpha}$ alone.

Duality: Solving the Dual (Step 1)

We want to solve: $\max_{\boldsymbol{\alpha} \in [0,1]^N} \min_{\boldsymbol{w}} \; G(\mathbf{w}, \boldsymbol{\alpha})$

- **Step 1: Minimize w.r.t. $\mathbf{w}$** (for a fixed $\boldsymbol{\alpha}$)

Duality: Solving the Dual (Step 1)

We want to solve: $\max_{\boldsymbol{\alpha} \in [0,1]^N} \min_{\boldsymbol{w}} \; G(\mathbf{w}, \boldsymbol{\alpha})$

- **Step 1: Minimize w.r.t. $\mathbf{w}$** (for a fixed $\boldsymbol{\alpha}$)
- We find the $\mathbf{w}$ that minimizes G by taking the gradient and setting it to zero:

$$\nabla_{\boldsymbol{w}} G(\mathbf{w}, \boldsymbol{\alpha}) = \nabla_{\boldsymbol{w}} \left(\frac{1}{N} \sum_{n=1}^{N} \alpha_n - \frac{1}{N} \sum_{n=1}^{N} \alpha_n y_n \mathbf{x}_n^\top \mathbf{w} + \tfrac{\lambda}{2} \|\mathbf{w}\|^2 \right)$$

$$= -\frac{1}{N} \sum_{n=1}^{N} \alpha_n y_n \mathbf{x}_n + \lambda \mathbf{w} \overset{\text{set}}{=} 0$$

Duality: Solving the Dual (Step 1)

We want to solve: $\max_{\boldsymbol{\alpha} \in [0,1]^N} \min_{\boldsymbol{w}} \; G(\mathbf{w}, \boldsymbol{\alpha})$

- **Step 1: Minimize w.r.t. $\mathbf{w}$** (for a fixed $\boldsymbol{\alpha}$)
- We find the $\mathbf{w}$ that minimizes G by taking the gradient and setting it to zero:

$$\begin{aligned}\nabla_{\boldsymbol{w}} G(\mathbf{w}, \boldsymbol{\alpha}) &= \nabla_{\boldsymbol{w}} \left(\frac{1}{N} \sum_{n=1}^{N} \alpha_n - \frac{1}{N} \sum_{n=1}^{N} \alpha_n y_n \mathbf{x}_n^\top \mathbf{w} + \tfrac{\lambda}{2} \|\mathbf{w}\|^2 \right) \\ &= -\frac{1}{N} \sum_{n=1}^{N} \alpha_n y_n \mathbf{x}_n + \lambda \mathbf{w} \overset{\text{set}}{=} 0\end{aligned}$$

- This gives us the optimal $\mathbf{w}$ *as a function of* $\boldsymbol{\alpha}$:

$$\mathbf{w}(\boldsymbol{\alpha}) = \tfrac{1}{\lambda N} \sum_{n=1}^{N} \alpha_n y_n \mathbf{x}_n = \tfrac{1}{\lambda N} \mathbf{X}^\top \mathbf{Y} \boldsymbol{\alpha}$$

Duality: Solving the Dual (Step 1)

We want to solve: $\max_{\boldsymbol{\alpha} \in [0,1]^N} \min_{\boldsymbol{w}} \; G(\mathbf{w}, \boldsymbol{\alpha})$

- **Step 1: Minimize w.r.t. $\mathbf{w}$** (for a fixed $\boldsymbol{\alpha}$)
- We find the $\mathbf{w}$ that minimizes G by taking the gradient and setting it to zero:

$$\nabla_{\boldsymbol{w}} G(\mathbf{w}, \boldsymbol{\alpha}) = \nabla_{\boldsymbol{w}} \left(\frac{1}{N} \sum_{n=1}^{N} \alpha_n - \frac{1}{N} \sum_{n=1}^{N} \alpha_n y_n \mathbf{x}_n^\top \mathbf{w} + \tfrac{\lambda}{2} \|\mathbf{w}\|^2 \right)$$

$$= -\frac{1}{N} \sum_{n=1}^{N} \alpha_n y_n \mathbf{x}_n + \lambda \mathbf{w} \overset{\text{set}}{=} 0$$

- This gives us the optimal $\mathbf{w}$ *as a function of* $\boldsymbol{\alpha}$:

$$\mathbf{w}(\boldsymbol{\alpha}) = \tfrac{1}{\lambda N} \sum_{n=1}^{N} \alpha_n y_n \mathbf{x}_n = \tfrac{1}{\lambda N} \mathbf{X}^\top \mathbf{Y} \boldsymbol{\alpha}$$

- **Crucial Insight:** The optimal weight vector $\mathbf{w}$ is a linear combination of the data points $\mathbf{x}_n$.

Duality: Solving the Dual (Step 2)

- **Step 2: Plug $\mathbf{w}(\alpha)$ back into $G(\mathbf{w}, \alpha)$.**

Duality: Solving the Dual (Step 2)

- **Step 2: Plug $\mathbf{w}(\boldsymbol{\alpha})$ back into $G(\mathbf{w}, \boldsymbol{\alpha})$.**
- We substitute $\mathbf{w}(\boldsymbol{\alpha}) = \frac{1}{\lambda N} \sum_{j=1}^{N} \alpha_j y_j \mathbf{x}_j$ into G:

$$G(\mathbf{w}(\boldsymbol{\alpha}), \boldsymbol{\alpha}) = \frac{1}{N} \sum_{n=1}^{N} \alpha_n (1 - y_n \mathbf{x}_n^\top \mathbf{w}(\boldsymbol{\alpha})) + \tfrac{\lambda}{2} \|\mathbf{w}(\boldsymbol{\alpha})\|^2$$

$$= \tfrac{1}{N} \sum_{n} \alpha_n - \tfrac{1}{N} \sum_{n} \alpha_n y_n \mathbf{x}_n^\top \left(\tfrac{1}{\lambda N} \sum_{j} \alpha_j y_j \mathbf{x}_j \right) + \tfrac{\lambda}{2} \left\| \tfrac{1}{\lambda N} \sum_{n} \alpha_n y_n \mathbf{x}_n \right\|^2$$

Duality: Solving the Dual (Step 2)

- **Step 2: Plug $\mathbf{w}(\boldsymbol{\alpha})$ back into $G(\mathbf{w}, \boldsymbol{\alpha})$.**
- We substitute $\mathbf{w}(\boldsymbol{\alpha}) = \frac{1}{\lambda N} \sum_{j=1}^{N} \alpha_j y_j \mathbf{x}_j$ into G:

$$G(\mathbf{w}(\boldsymbol{\alpha}), \boldsymbol{\alpha}) = \frac{1}{N} \sum_{n=1}^{N} \alpha_n (1 - y_n \mathbf{x}_n^\top \mathbf{w}(\boldsymbol{\alpha})) + \tfrac{\lambda}{2} \|\mathbf{w}(\boldsymbol{\alpha})\|^2$$

$$= \tfrac{1}{N} \sum_n \alpha_n - \tfrac{1}{N} \sum_n \alpha_n y_n \mathbf{x}_n^\top \left(\tfrac{1}{\lambda N} \sum_j \alpha_j y_j \mathbf{x}_j\right) + \tfrac{\lambda}{2} \left\| \tfrac{1}{\lambda N} \sum_n \alpha_n y_n \mathbf{x}_n \right\|^2$$

- Let's simplify the terms. Note $\sum_n \sum_j \alpha_n \alpha_j y_n y_j (\mathbf{x}_n^\top \mathbf{x}_j) = \boldsymbol{\alpha}^\top \mathbf{Y}\mathbf{K}\mathbf{Y}\boldsymbol{\alpha}$ where $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.

$$= \tfrac{1}{N} \boldsymbol{\alpha}^\top \mathbf{1} - \tfrac{1}{\lambda N^2} (\boldsymbol{\alpha}^\top \mathbf{Y}\mathbf{X}\mathbf{X}^\top \mathbf{Y}\boldsymbol{\alpha}) + \tfrac{\lambda}{2\lambda^2 N^2} (\boldsymbol{\alpha}^\top \mathbf{Y}\mathbf{X}\mathbf{X}^\top \mathbf{Y}\boldsymbol{\alpha})$$

$$= \tfrac{1}{N} \boldsymbol{\alpha}^\top \mathbf{1} - \tfrac{1}{2\lambda N^2} \boldsymbol{\alpha}^\top \mathbf{Y}\mathbf{K}\mathbf{Y}\boldsymbol{\alpha}$$

Duality: Solving the Dual (Step 2)

- **Step 2: Plug $\mathbf{w}(\boldsymbol{\alpha})$ back into $G(\mathbf{w}, \boldsymbol{\alpha})$.**
- We substitute $\mathbf{w}(\boldsymbol{\alpha}) = \frac{1}{\lambda N} \sum_{j=1}^{N} \alpha_j y_j \mathbf{x}_j$ into G:

$$G(\mathbf{w}(\boldsymbol{\alpha}), \boldsymbol{\alpha}) = \frac{1}{N} \sum_{n=1}^{N} \alpha_n (1 - y_n \mathbf{x}_n^\top \mathbf{w}(\boldsymbol{\alpha})) + \tfrac{\lambda}{2} \|\mathbf{w}(\boldsymbol{\alpha})\|^2$$

$$= \tfrac{1}{N} \sum_n \alpha_n - \tfrac{1}{N} \sum_n \alpha_n y_n \mathbf{x}_n^\top \left(\tfrac{1}{\lambda N} \sum_j \alpha_j y_j \mathbf{x}_j \right) + \tfrac{\lambda}{2} \left\| \tfrac{1}{\lambda N} \sum_n \alpha_n y_n \mathbf{x}_n \right\|^2$$

- Let's simplify the terms. Note $\sum_n \sum_j \alpha_n \alpha_j y_n y_j (\mathbf{x}_n^\top \mathbf{x}_j) = \boldsymbol{\alpha}^\top \mathbf{Y}\mathbf{K}\mathbf{Y}\boldsymbol{\alpha}$ where $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.

$$= \tfrac{1}{N} \boldsymbol{\alpha}^\top \mathbf{1} - \tfrac{1}{\lambda N^2} (\boldsymbol{\alpha}^\top \mathbf{Y}\mathbf{X}\mathbf{X}^\top \mathbf{Y}\boldsymbol{\alpha}) + \tfrac{\lambda}{2\lambda^2 N^2} (\boldsymbol{\alpha}^\top \mathbf{Y}\mathbf{X}\mathbf{X}^\top \mathbf{Y}\boldsymbol{\alpha})$$

$$= \tfrac{1}{N} \boldsymbol{\alpha}^\top \mathbf{1} - \tfrac{1}{2\lambda N^2} \boldsymbol{\alpha}^\top \mathbf{Y}\mathbf{K}\mathbf{Y}\boldsymbol{\alpha}$$

- This gives the final Dual Problem:

$$\max_{\boldsymbol{\alpha} \in [0,1]^N} \ \tfrac{1}{N} \boldsymbol{\alpha}^\top \mathbf{1} - \tfrac{1}{2\lambda N^2} \boldsymbol{\alpha}^\top \mathbf{Q}\boldsymbol{\alpha}$$

where $\mathbf{Q} = \mathbf{Y}\mathbf{K}\mathbf{Y} = \operatorname{diag}(\mathbf{y})\mathbf{X}\mathbf{X}^\top \operatorname{diag}(\mathbf{y})$

Duality: Why Bother?

The dual problem is often better:

$$\max_{\boldsymbol{\alpha} \in [0,1]^N} \frac{1}{N} \boldsymbol{\alpha}^\top \mathbf{1} - \frac{1}{2\lambda N^2} \boldsymbol{\alpha}^\top \mathbf{Q} \boldsymbol{\alpha} \quad \text{where } \mathbf{Q} = \mathbf{Y}(\mathbf{X}\mathbf{X}^\top)\mathbf{Y}$$

- **1. It's a constrained quadratic problem.** We can solve it efficiently with Coordinate Ascent.

Duality: Why Bother?

The dual problem is often better:

$$\max_{\boldsymbol{\alpha} \in [0,1]^N} \frac{1}{N} \boldsymbol{\alpha}^\top \mathbf{1} - \frac{1}{2\lambda N^2} \boldsymbol{\alpha}^\top \mathbf{Q} \boldsymbol{\alpha} \quad \text{where } \mathbf{Q} = \mathbf{Y}(\mathbf{X}\mathbf{X}^\top)\mathbf{Y}$$

- **1. It's a constrained quadratic problem.** We can solve it efficiently with Coordinate Ascent.
- **2. The data only appears as $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.**
  - This $N \times N$ matrix is called the Kernel.
  - The problem complexity now depends on N (samples), not D (features).
  - **This is a huge win when $D \gg N$ (e.g., text, genomics).**
  - This property is what enables the Kernel Trick.

Duality: Why Bother?

The dual problem is often better:

$$\max_{\boldsymbol{\alpha} \in [0,1]^N} \frac{1}{N} \boldsymbol{\alpha}^\top \mathbf{1} - \frac{1}{2\lambda N^2} \boldsymbol{\alpha}^\top \mathbf{Q} \boldsymbol{\alpha} \quad \text{where } \mathbf{Q} = \mathbf{Y}(\mathbf{X}\mathbf{X}^\top)\mathbf{Y}$$

- **1. It's a constrained quadratic problem.** We can solve it efficiently with Coordinate Ascent.
- **2. The data only appears as $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.**
  - This $N \times N$ matrix is called the Kernel.
  - The problem complexity now depends on N (samples), not D (features).
  - **This is a huge win when $D \gg N$ (e.g., text, genomics).**
  - This property is what enables the Kernel Trick.
- **3. Many example weights can be zero.**
  - Points strictly outside the margin have $\alpha_n = 0$; many such points can yield a sparse solution.
  - The $\mathbf{w}$ vector ($\mathbf{w} = \frac{1}{\lambda N} \sum \alpha_n y_n \mathbf{x}_n$) only depends on the points $\mathbf{x}_n$ where $\alpha_n > 0$.
  - These points are the Support Vectors.

What Does a Sample's Optimal Weight Tell Us?

At an optimal primal-dual pair, write $m_n = y_n \mathbf{x}_n^\top \mathbf{w}^*$. Recall how each hinge term was represented:

$$[1 - m_n]_+ = \max_{\alpha_n \in [0,1]} \alpha_n (1 - m_n).$$

What Does a Sample's Optimal Weight Tell Us?

At an optimal primal-dual pair, write $m_n = y_n \mathbf{x}_n^\top \mathbf{w}^*$. Recall how each hinge term was represented:

$$[1 - m_n]_+ = \max_{\alpha_n \in [0,1]} \alpha_n (1 - m_n).$$

The sign of $1 - m_n$ determines the maximizing weight.

Position	Inner maximization	Required α_n^*
$m_n > 1$	Negative coefficient: choose zero	0
$m_n = 1$	Zero coefficient: all values tie	Any value in $[0, 1]$
$m_n < 1$	Positive coefficient: choose one	1

What Does a Sample's Optimal Weight Tell Us?

At an optimal primal-dual pair, write $m_n = y_n \mathbf{x}_n^\top \mathbf{w}^*$. Recall how each hinge term was represented:

$$[1 - m_n]_+ = \max_{\alpha_n \in [0,1]} \alpha_n (1 - m_n).$$

The sign of $1 - m_n$ determines the maximizing weight.

Position	Inner maximization	Required α_n^*
$m_n > 1$	Negative coefficient: choose zero	0
$m_n = 1$	Zero coefficient: all values tie	Any value in $[0, 1]$
$m_n < 1$	Positive coefficient: choose one	1

At the margin When $m_n = 1$, the hinge term alone does not determine α_n^*. All the weights must also be consistent with the **same optimal classifier**.

Which Training Examples Are Support Vectors?

Weights enter the fitted expansion; the plots below use functional margin m:

$$\mathbf{w}^* = \frac{1}{\lambda N} \sum_n \alpha_n^* y_n \mathbf{x}_n.$$

Outside the margin



$m_n > 1$

$\alpha_n^* = 0$

Which Training Examples Are Support Vectors?

Weights enter the fitted expansion; the plots below use functional margin m:

$$\mathbf{w}^* = \frac{1}{\lambda N} \sum_n \alpha_n^* y_n \mathbf{x}_n.$$

Outside the margin



$m_n > 1$

$\alpha_n^* = 0$

On the margin



$m_n = 1$

$0 \leq \alpha_n^* \leq 1$

Which Training Examples Are Support Vectors?

Weights enter the fitted expansion; the plots below use functional margin m:

$$\mathbf{w}^* = \frac{1}{\lambda N} \sum_n \alpha_n^* y_n \mathbf{x}_n.$$

Outside the margin



$m_n > 1$

$\alpha_n^* = 0$

On the margin



$m_n = 1$

$0 \leq \alpha_n^* \leq 1$

Margin violation



$m_n < 1$

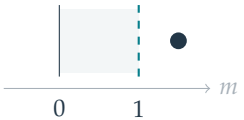
$\alpha_n^* = 1$

Which Training Examples Are Support Vectors?

Weights enter the fitted expansion; the plots below use functional margin m:

$$\mathbf{w}^* = \frac{1}{\lambda N} \sum_n \alpha_n^* y_n \mathbf{x}_n.$$

Outside the margin



$m_n > 1$

$\alpha_n^* = 0$

On the margin



$m_n = 1$

$0 \leq \alpha_n^* \leq 1$

Margin violation



$m_n < 1$

$\alpha_n^* = 1$

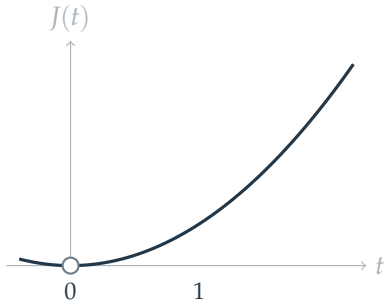
Next: check optimality Pointwise rules are not enough: the classifier and all sample weights must be **jointly optimal**. How do we check this?

How Do We Check Optimality with Constraints?

Start with the unconstrained test:

$$\min_t J(t) = \tfrac{1}{2}t^2$$

has optimum $t^* = 0$, where $J'(0) = 0$.



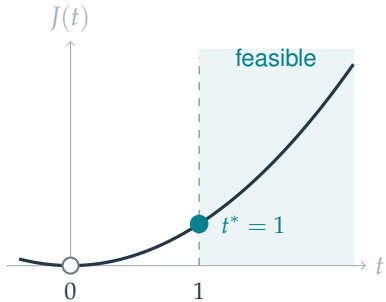
How Do We Check Optimality with Constraints?

Start with the unconstrained test:

$$\min_t J(t) = \tfrac{1}{2}t^2$$

has optimum $t^* = 0$, where $J'(0) = 0$.

Now require $t \geq 1$. The point 0 is no longer allowed.
The best feasible point is $t^* = 1$.



How Do We Check Optimality with Constraints?

Start with the unconstrained test:

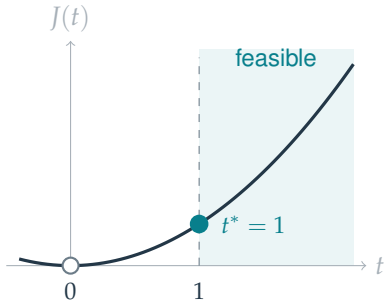
$$\min_t J(t) = \tfrac{1}{2}t^2$$

has optimum $t^* = 0$, where $J'(0) = 0$.

Now require $t \geq 1$. The point 0 is no longer allowed. The best feasible point is $t^* = 1$.

But $J'(1) = 1 \neq 0$.

Decreasing t would lower the cost, but would violate the constraint.



How Do We Check Optimality with Constraints?

Start with the unconstrained test:

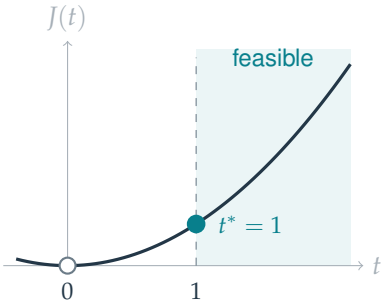
$$\min_t J(t) = \tfrac{1}{2}t^2$$

has optimum $t^* = 0$, where $J'(0) = 0$.

Now require $t \geq 1$. The point 0 is no longer allowed. The best feasible point is $t^* = 1$.

But $J'(1) = 1 \neq 0$.

Decreasing t would lower the cost, but would violate the constraint.



What is missing from the zero-gradient test? At a boundary optimum, the constraints can block a direction that would reduce the objective. We need optimality conditions that account for those constraints.

What Are the KKT Conditions?

Karush–Kuhn–Tucker (KKT) conditions are first-order conditions for optimality in a **constrained optimization problem**:

$$\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \qquad \text{s.t.} \quad c_i(\boldsymbol{\theta}) \geq 0.$$

What Are the KKT Conditions?

Karush–Kuhn–Tucker (KKT) conditions are first-order conditions for optimality in a **constrained optimization problem**:

$$\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \qquad \text{s.t.} \quad c_i(\boldsymbol{\theta}) \geq 0.$$

Introduce constraint weights $\mu_i \geq 0$ and the Lagrangian:

$$L(\boldsymbol{\theta}, \boldsymbol{\mu}) = J(\boldsymbol{\theta}) - \sum_i \mu_i c_i(\boldsymbol{\theta}).$$

What Are the KKT Conditions?

Karush–Kuhn–Tucker (KKT) conditions are first-order conditions for optimality in a **constrained optimization problem**:

$$\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \qquad \text{s.t.} \quad c_i(\boldsymbol{\theta}) \geq 0.$$

Introduce constraint weights $\mu_i \geq 0$ and the Lagrangian:

$$L(\boldsymbol{\theta}, \boldsymbol{\mu}) = J(\boldsymbol{\theta}) - \sum_i \mu_i c_i(\boldsymbol{\theta}).$$

Primal feasibility $c_i(\boldsymbol{\theta}) \geq 0$: obey the constraints.

What Are the KKT Conditions?

Karush–Kuhn–Tucker (KKT) conditions are first-order conditions for optimality in a **constrained optimization problem**:

$$\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \qquad \text{s.t.} \quad c_i(\boldsymbol{\theta}) \geq 0.$$

Introduce constraint weights $\mu_i \geq 0$ and the Lagrangian:

$$L(\boldsymbol{\theta}, \boldsymbol{\mu}) = J(\boldsymbol{\theta}) - \sum_i \mu_i c_i(\boldsymbol{\theta}).$$

Primal feasibility	$c_i(\boldsymbol{\theta}) \geq 0$: obey the constraints.
Dual feasibility	$\mu_i \geq 0$: nonnegative constraint weights.

What Are the KKT Conditions?

Karush–Kuhn–Tucker (KKT) conditions are first-order conditions for optimality in a **constrained optimization problem**:

$$\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \qquad \text{s.t.} \quad c_i(\boldsymbol{\theta}) \geq 0.$$

Introduce constraint weights $\mu_i \geq 0$ and the Lagrangian:

$$L(\boldsymbol{\theta}, \boldsymbol{\mu}) = J(\boldsymbol{\theta}) - \sum_i \mu_i c_i(\boldsymbol{\theta}).$$

Primal feasibility	$c_i(\boldsymbol{\theta}) \geq 0$: obey the constraints.
Dual feasibility	$\mu_i \geq 0$: nonnegative constraint weights.
Stationarity	$\nabla_{\boldsymbol{\theta}} L = 0$: balance objective and constraint gradients.

What Are the KKT Conditions?

Karush–Kuhn–Tucker (KKT) conditions are first-order conditions for optimality in a **constrained optimization problem**:

$$\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \qquad \text{s.t.} \quad c_i(\boldsymbol{\theta}) \geq 0.$$

Introduce constraint weights $\mu_i \geq 0$ and the Lagrangian:

$$L(\boldsymbol{\theta}, \boldsymbol{\mu}) = J(\boldsymbol{\theta}) - \sum_i \mu_i c_i(\boldsymbol{\theta}).$$

Primal feasibility	$c_i(\boldsymbol{\theta}) \geq 0$: obey the constraints.
Dual feasibility	$\mu_i \geq 0$: nonnegative constraint weights.
Stationarity	$\nabla_{\boldsymbol{\theta}} L = 0$: balance objective and constraint gradients.
Complementary slackness	$\mu_i c_i(\boldsymbol{\theta}) = 0$: a positive weight requires a tight constraint.

What does KKT give us? A way to check a candidate solution together with its constraint weights. For convex problems such as our SVM, satisfying KKT certifies a **global optimum**.

Apply KKT to the Boundary Optimum

For $\min_t \frac{1}{2}t^2$ subject to $t \geq 1$, use $c(t) = t - 1$:

$$L(t, \mu) = \tfrac{1}{2}t^2 - \mu(t - 1).$$

Apply KKT to the Boundary Optimum

For $\min_t \frac{1}{2}t^2$ subject to $t \geq 1$, use $c(t) = t - 1$:

$$L(t, \mu) = \tfrac{1}{2}t^2 - \mu(t - 1).$$

Primal feasibility	$t \geq 1$
Dual feasibility	$\mu \geq 0$
Stationarity	$\partial L/\partial t = t - \mu = 0$
Complementary slackness	$\mu(t - 1) = 0$

Apply KKT to the Boundary Optimum

For $\min_t \frac{1}{2}t^2$ subject to $t \geq 1$, use $c(t) = t - 1$:

$$L(t, \mu) = \tfrac{1}{2}t^2 - \mu(t - 1).$$

Primal feasibility	$t \geq 1$
Dual feasibility	$\mu \geq 0$
Stationarity	$\partial L/\partial t = t - \mu = 0$
Complementary slackness	$\mu(t - 1) = 0$

Stationarity gives $\mu = t \geq 1$, hence $\mu > 0$. Complementary slackness then forces $t - 1 = 0$: Thus $t^* = 1$ and $\mu^* = 1$.

Apply KKT to the Boundary Optimum

For $\min_t \frac{1}{2}t^2$ subject to $t \geq 1$, use $c(t) = t - 1$:

$$L(t, \mu) = \tfrac{1}{2}t^2 - \mu(t - 1).$$

Primal feasibility	$t \geq 1$
Dual feasibility	$\mu \geq 0$
Stationarity	$\partial L/\partial t = t - \mu = 0$
Complementary slackness	$\mu(t - 1) = 0$

Stationarity gives $\mu = t \geq 1$, hence $\mu > 0$. Complementary slackness then forces $t - 1 = 0$: Thus $t^* = 1$ and $\mu^* = 1$.

Balanced gradients At $t = 1$, $J'(t) = 1$, but $\partial L/\partial t = 1 - 1 = 0$. The multiplier balances the objective and constraint gradients.

Apply the KKT Checklist to Soft-Margin SVM

Now the primal variables are $(\mathbf{w}, \boldsymbol{\xi})$, and our scaled dual weights are $\boldsymbol{\alpha}$. Here $m_n = y_n \mathbf{x}_n^\top \mathbf{w}$.

Apply the KKT Checklist to Soft-Margin SVM

Now the primal variables are $(\mathbf{w}, \boldsymbol{\xi})$, and our scaled dual weights are $\boldsymbol{\alpha}$. Here $m_n = y_n \mathbf{x}_n^\top \mathbf{w}$.

1. **Primal feasibility:** the classifier and slacks obey $m_n + \xi_n \geq 1$ and $\xi_n \geq 0$.

Apply the KKT Checklist to Soft-Margin SVM

Now the primal variables are $(\mathbf{w}, \boldsymbol{\xi})$, and our scaled dual weights are $\boldsymbol{\alpha}$. Here $m_n = y_n \mathbf{x}_n^\top \mathbf{w}$.

1. **Primal feasibility:** the classifier and slacks obey $m_n + \xi_n \geq 1$ and $\xi_n \geq 0$.
2. **Dual feasibility:** every weight obeys $0 \leq \alpha_n \leq 1$.

Apply the KKT Checklist to Soft-Margin SVM

Now the primal variables are $(\mathbf{w}, \boldsymbol{\xi})$, and our scaled dual weights are $\boldsymbol{\alpha}$. Here $m_n = y_n \mathbf{x}_n^\top \mathbf{w}$.

1. **Primal feasibility:** the classifier and slacks obey $m_n + \xi_n \geq 1$ and $\xi_n \geq 0$.
2. **Dual feasibility:** every weight obeys $0 \leq \alpha_n \leq 1$.
3. **Stationarity:** the classifier matches the sample weights:

$$\lambda \mathbf{w} = \frac{1}{N} \sum_n \alpha_n y_n \mathbf{x}_n.$$

Apply the KKT Checklist to Soft-Margin SVM

Now the primal variables are $(\mathbf{w}, \boldsymbol{\xi})$, and our scaled dual weights are $\boldsymbol{\alpha}$. Here $m_n = y_n \mathbf{x}_n^\top \mathbf{w}$.

1. **Primal feasibility:** the classifier and slacks obey $m_n + \xi_n \geq 1$ and $\xi_n \geq 0$.
2. **Dual feasibility:** every weight obeys $0 \leq \alpha_n \leq 1$.
3. **Stationarity:** the classifier matches the sample weights:

$$\lambda \mathbf{w} = \frac{1}{N} \sum_n \alpha_n y_n \mathbf{x}_n.$$

4. **Complementary slackness:** unused room in a constraint cannot carry a positive multiplier. We unpack this next.

What the checklist certifies here For this convex soft-margin SVM, a feasible pair satisfying these conditions is optimal. KKT checks the **primal and dual together**.

Why Must the Complementary Products Be Zero?

To unpack complementary slackness, define the margin **constraint gap**

$$s_n = m_n + \xi_n - 1 \geq 0.$$

$s_n = 0$ means the margin constraint is **tight**; $s_n > 0$ means unused room.

Why Must the Complementary Products Be Zero?

To unpack complementary slackness, define the margin **constraint gap**

$$s_n = m_n + \xi_n - 1 \geq 0.$$

$s_n = 0$ means the margin constraint is **tight**; $s_n > 0$ means unused room.
Let $P(\mathbf{w}, \boldsymbol{\xi}) = \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{N} \sum_n \xi_n$. Subtract the previously defined G:

$$P(\mathbf{w}, \boldsymbol{\xi}) - G(\mathbf{w}, \boldsymbol{\alpha}) = \frac{1}{N} \sum_n \left[\underbrace{(1 - \alpha_n)\xi_n}_{\geq 0} + \underbrace{\alpha_n s_n}_{\geq 0} \right].$$

Why Must the Complementary Products Be Zero?

To unpack complementary slackness, define the margin **constraint gap**

$$s_n = m_n + \xi_n - 1 \geq 0.$$

$s_n = 0$ means the margin constraint is **tight**; $s_n > 0$ means unused room.
Let $P(\mathbf{w}, \boldsymbol{\xi}) = \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{N} \sum_n \xi_n$. Subtract the previously defined G:

$$P(\mathbf{w}, \boldsymbol{\xi}) - G(\mathbf{w}, \boldsymbol{\alpha}) = \frac{1}{N} \sum_n \left[\underbrace{(1 - \alpha_n)\xi_n}_{\geq 0} + \underbrace{\alpha_n s_n}_{\geq 0} \right].$$

At a matching optimal pair, strong duality makes this gap zero. A sum of nonnegative terms can be zero only when each term is zero.

Why Must the Complementary Products Be Zero?

To unpack complementary slackness, define the margin **constraint gap**

$$s_n = m_n + \xi_n - 1 \geq 0.$$

$s_n = 0$ means the margin constraint is **tight**; $s_n > 0$ means unused room.
Let $P(\mathbf{w}, \boldsymbol{\xi}) = \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{N} \sum_n \xi_n$. Subtract the previously defined G:

$$P(\mathbf{w}, \boldsymbol{\xi}) - G(\mathbf{w}, \boldsymbol{\alpha}) = \frac{1}{N} \sum_n \left[\underbrace{(1 - \alpha_n)\xi_n}_{\geq 0} + \underbrace{\alpha_n s_n}_{\geq 0} \right].$$

At a matching optimal pair, strong duality makes this gap zero. A sum of nonnegative terms can be zero only when each term is zero.

Complementary slackness $\quad \alpha_n s_n = 0, \qquad (1 - \alpha_n)\xi_n = 0 \quad$ for every n.

Use the KKT Products to Check the Three Cases

Use the smallest feasible slack $\xi_n^* = [1 - m_n]_+$ and $s_n = m_n + \xi_n^* - 1$.

$$\alpha_n^* s_n = 0, \qquad (1 - \alpha_n^*)\xi_n^* = 0.$$

Use the KKT Products to Check the Three Cases

Use the smallest feasible slack $\xi_n^* = [1 - m_n]_+$ and $s_n = m_n + \xi_n^* - 1$.

$$\alpha_n^* s_n = 0, \qquad (1 - \alpha_n^*)\xi_n^* = 0.$$

Outside the margin: $m_n > 1$**.** Here $\xi_n^* = 0$ and $s_n = m_n - 1 > 0$, so the first product forces $\alpha_n^* = 0$.

Use the KKT Products to Check the Three Cases

Use the smallest feasible slack $\xi_n^* = [1 - m_n]_+$ and $s_n = m_n + \xi_n^* - 1$.

$$\alpha_n^* s_n = 0, \qquad (1 - \alpha_n^*)\xi_n^* = 0.$$

Outside the margin: $m_n > 1$**.** Here $\xi_n^* = 0$ and $s_n = m_n - 1 > 0$, so the first product forces $\alpha_n^* = 0$.

Violating the margin: $m_n < 1$**.** Here $\xi_n^* = 1 - m_n > 0$ and $s_n = 0$, so the second product forces $\alpha_n^* = 1$.

Use the KKT Products to Check the Three Cases

Use the smallest feasible slack $\xi_n^* = [1 - m_n]_+$ and $s_n = m_n + \xi_n^* - 1$.

$$\alpha_n^* s_n = 0, \qquad (1 - \alpha_n^*)\xi_n^* = 0.$$

Outside the margin: $m_n > 1$. Here $\xi_n^* = 0$ and $s_n = m_n - 1 > 0$, so the first product forces $\alpha_n^* = 0$.

Violating the margin: $m_n < 1$. Here $\xi_n^* = 1 - m_n > 0$ and $s_n = 0$, so the second product forces $\alpha_n^* = 1$.

Exactly on the margin: $m_n = 1$. Both $\xi_n^* = 0$ and $s_n = 0$. The products impose no further restriction beyond $\alpha_n^* \in [0, 1]$; stationarity still has to hold.

Now verify the borderline case A tight constraint need not have a positive weight. Next, use duplicate samples to check that $m_n = 1$ and $\alpha_n^* = 0$ can occur in an **optimal** solution.

Check the Zero-Weight Case with Duplicate Samples

A one-dimensional example: $b = 0$, $N = 3$, $\lambda = 2/3$. Points A and B are identical. The classifier $f(x) = x$ has $w = 1$.

Point	x_n	y_n	$m_n = y_n w x_n$
A	$+1$	$+1$	1
B	$+1$	$+1$	1
C	-1	-1	1

All three points are **exactly on their margin boundaries**.

Check the Zero-Weight Case with Duplicate Samples

A one-dimensional example: $b = 0$, $N = 3$, $\lambda = 2/3$. Points A and B are identical. The classifier $f(x) = x$ has $w = 1$.

Point	x_n	y_n	$m_n = y_n w x_n$	Weights I
A	$+1$	$+1$	1	0
B	$+1$	$+1$	1	1
C	-1	-1	1	1

All three points are **exactly on their margin boundaries**.
Since $\lambda N = 2$ and every $y_n x_n = 1$, weights $(0, 1, 1)$ reconstruct

$$w = \frac{1}{\lambda N} \sum_n \alpha_n y_n x_n = \frac{0 + 1 + 1}{2} = 1.$$

Point A has $m_A = 1$ but $\alpha_A = 0$.

Check the Zero-Weight Case with Duplicate Samples

A one-dimensional example: $b = 0$, $N = 3$, $\lambda = 2/3$. Points A and B are identical. The classifier $f(x) = x$ has $w = 1$.

Point	x_n	y_n	$m_n = y_n w x_n$	Weights I	Weights II
A	$+1$	$+1$	1	0	1
B	$+1$	$+1$	1	1	0
C	-1	-1	1	1	1

All three points are **exactly on their margin boundaries**.
Since $\lambda N = 2$ and every $y_n x_n = 1$, weights $(0, 1, 1)$ reconstruct

$$w = \frac{1}{\lambda N} \sum_n \alpha_n y_n x_n = \frac{0 + 1 + 1}{2} = 1.$$

Point A has $m_A = 1$ but $\alpha_A = 0$.
Swapping A and B's weights gives $(1, 0, 1)$ and the **same** $w = 1$. The duplicate points can take each other's place in this expansion.

Check the Zero-Weight Case with Duplicate Samples

A one-dimensional example: $b = 0$, $N = 3$, $\lambda = 2/3$. Points A and B are identical. The classifier $f(x) = x$ has $w = 1$.

Point	x_n	y_n	$m_n = y_n w x_n$	Weights I	Weights II
A	$+1$	$+1$	1	0	1
B	$+1$	$+1$	1	1	0
C	-1	-1	1	1	1

All three points are **exactly on their margin boundaries**.
Since $\lambda N = 2$ and every $y_n x_n = 1$, weights $(0, 1, 1)$ reconstruct

$$w = \frac{1}{\lambda N} \sum_n \alpha_n y_n x_n = \frac{0 + 1 + 1}{2} = 1.$$

Point A has $m_A = 1$ but $\alpha_A = 0$.
Swapping A and B's weights gives $(1, 0, 1)$ and the **same** $w = 1$. The duplicate points can take each other's place in this expansion.
KKT check: both weight vectors satisfy all KKT conditions and are optimal. The classifier is unchanged.

Optimization: Coordinate Ascent

Now compute the weights. Maximize the dual $g(\boldsymbol{\alpha}) = \frac{1}{N}\boldsymbol{\alpha}^\top \mathbf{1} - \frac{1}{2\lambda N^2}\boldsymbol{\alpha}^\top \mathbf{Q}\boldsymbol{\alpha}$ subject to $\alpha_n \in [0,1]$.

Idea: Update one coordinate at a time, keeping others fixed.

1. Initialize $\boldsymbol{\alpha}^{(0)} = \mathbf{0}$.
2. Sample a coordinate $n \in \{1, \ldots, N\}$.
3. Keep all other coordinates fixed and solve

$$u^\star = \arg\max_{u \in [0,1]} g(\alpha_1^{(t)}, \ldots, \alpha_{n-1}^{(t)}, u, \alpha_{n+1}^{(t)}, \ldots, \alpha_N^{(t)}).$$

   This is a one-dimensional concave quadratic on an interval.

4. Set $\alpha_n^{(t+1)} = u^\star$; leave other coordinates unchanged.
5. Repeat until convergence.

Issues with SVM

- There is no obvious probabilistic interpretation (unlike Logistic Regression).
  - The output is a "score" $\mathbf{x}^\top \mathbf{w}$, not a probability $p(y = 1|\mathbf{x})$.
  - (Though this can be mitigated with "Platt Scaling").

Issues with SVM

- There is no obvious probabilistic interpretation (unlike Logistic Regression).
  - The output is a "score" $\mathbf{x}^\top \mathbf{w}$, not a probability $p(y = 1|\mathbf{x})$.
  - (Though this can be mitigated with "Platt Scaling").
- Extension to multi-class classification is non-trivial. Common approaches:
  - **One-vs-Rest (OvR):** Train K binary classifiers (class k vs. all others).
  - **One-vs-One (OvO):** Train $\frac{K(K-1)}{2}$ binary classifiers (class i vs. class j).

Motivation: Why Kernelize?

- We just saw that the SVM dual depends only on inner products $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.

Motivation: Why Kernelize?

- We just saw that the SVM dual depends only on inner products $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.
- This $N \times N$ matrix is called a kernel matrix.

Motivation: Why Kernelize?

- We just saw that the SVM dual depends only on inner products $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.
- This $N \times N$ matrix is called a kernel matrix.
- Let's see if this property holds for other algorithms, like Ridge Regression.

Motivation: Why Kernelize?

- We just saw that the SVM dual depends only on inner products $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.
- This $N \times N$ matrix is called a kernel matrix.
- Let's see if this property holds for other algorithms, like Ridge Regression.
- This will lead us to a powerful general idea: the Kernel Trick.

Motivation: Why Kernelize?

- We just saw that the SVM dual depends only on inner products $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.
- This $N \times N$ matrix is called a kernel matrix.
- Let's see if this property holds for other algorithms, like Ridge Regression.
- This will lead us to a powerful general idea: the Kernel Trick.
- This trick allows us to use highly complex, non-linear, and even infinite-dimensional feature maps *without* paying the computational cost.

Alternative Formulation of Ridge Regression

Recall the **primal** problem for Ridge Regression:

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

Alternative Formulation of Ridge Regression

Recall the **primal** problem for Ridge Regression:

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

The solution $\mathbf{w}^\star \in \mathbb{R}^D$ is:

$$\mathbf{w}^\star = \frac{1}{N}(\frac{1}{N}\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}_D)^{-1}\mathbf{X}^\top\mathbf{y}$$

Alternative Formulation of Ridge Regression

Recall the **primal** problem for Ridge Regression:

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

The solution $\mathbf{w}^\star \in \mathbb{R}^D$ is:

$$\mathbf{w}^\star = \frac{1}{N}(\frac{1}{N}\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}_D)^{-1}\mathbf{X}^\top\mathbf{y}$$

- The main computation is inverting a $D \times D$ matrix.

Alternative Formulation of Ridge Regression

Recall the **primal** problem for Ridge Regression:

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

The solution $\mathbf{w}^\star \in \mathbb{R}^D$ is:

$$\mathbf{w}^\star = \frac{1}{N}(\frac{1}{N}\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}_D)^{-1}\mathbf{X}^\top\mathbf{y}$$

- The main computation is inverting a $D \times D$ matrix.
- This costs $O(D^3 + ND^2)$. This is great if $D \ll N$.

Alternative Formulation of Ridge Regression

Recall the **primal** problem for Ridge Regression:

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

The solution $\mathbf{w}^\star \in \mathbb{R}^D$ is:

$$\mathbf{w}^\star = \frac{1}{N}(\frac{1}{N}\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}_D)^{-1}\mathbf{X}^\top\mathbf{y}$$

- The main computation is inverting a $D \times D$ matrix.
- This costs $O(D^3 + ND^2)$. This is great if $D \ll N$.
- But what if $D \gg N$? (e.g., 100 samples, 1,000,000 features)

Alternative Formulation (Dual)

We claim the solution can be written alternatively as:

$$\mathbf{w}^\star = \tfrac{1}{N}\mathbf{X}^\top(\tfrac{1}{N}\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_N)^{-1}\mathbf{y} \tag{3}$$

Alternative Formulation (Dual)

We claim the solution can be written alternatively as:

$$\mathbf{w}^\star = \tfrac{1}{N}\mathbf{X}^\top(\tfrac{1}{N}\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_N)^{-1}\mathbf{y} \tag{3}$$

- The main computation is now inverting an $N \times N$ matrix.

Alternative Formulation (Dual)

We claim the solution can be written alternatively as:

$$\mathbf{w}^\star = \tfrac{1}{N}\mathbf{X}^\top(\tfrac{1}{N}\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_N)^{-1}\mathbf{y} \tag{3}$$

- The main computation is now inverting an $N \times N$ matrix.
- This costs $O(N^3 + DN^2)$.

Alternative Formulation (Dual)

We claim the solution can be written alternatively as:

$$\mathbf{w}^\star = \tfrac{1}{N}\mathbf{X}^\top(\tfrac{1}{N}\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_N)^{-1}\mathbf{y} \tag{3}$$

- The main computation is now inverting an $N \times N$ matrix.
- This costs $O(N^3 + DN^2)$.
- This is much better if $N \ll D$!

Alternative Formulation (Dual)

We claim the solution can be written alternatively as:

$$\mathbf{w}^\star = \tfrac{1}{N}\mathbf{X}^\top(\tfrac{1}{N}\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_N)^{-1}\mathbf{y} \tag{3}$$

- The main computation is now inverting an $N \times N$ matrix.
- This costs $O(N^3 + DN^2)$.
- This is much better if $N \ll D$!

This relies on the "push-through" matrix identity:

$$(\mathbf{P}\mathbf{Q} + \mathbf{I}_N)^{-1}\mathbf{P} = \mathbf{P}(\mathbf{Q}\mathbf{P} + \mathbf{I}_M)^{-1}$$

(Here, $\mathbf{P} = \mathbf{X}^\top$ and $\mathbf{Q} = \frac{1}{\lambda N}\mathbf{X}$)

Proof of Alternative Formulation

- This relies on a common matrix identity (sometimes called the “push-through” identity).

Proof of Alternative Formulation

- This relies on a common matrix identity (sometimes called the “push-through” identity).
- Let $\mathbf{P} \in \mathbb{R}^{m \times n}$ and $\mathbf{Q} \in \mathbb{R}^{n \times m}$.

Proof of Alternative Formulation

- This relies on a common matrix identity (sometimes called the “push-through” identity).
- Let $\mathbf{P} \in \mathbb{R}^{m \times n}$ and $\mathbf{Q} \in \mathbb{R}^{n \times m}$.
- We have: $\mathbf{P}(\mathbf{QP} + \mathbf{I}_n) = \mathbf{PQP} + \mathbf{P} = (\mathbf{PQ} + \mathbf{I}_m)\mathbf{P}$.

Proof of Alternative Formulation

- This relies on a common matrix identity (sometimes called the “push-through” identity).
- Let $\mathbf{P} \in \mathbb{R}^{m \times n}$ and $\mathbf{Q} \in \mathbb{R}^{n \times m}$.
- We have: $\mathbf{P}(\mathbf{QP} + \mathbf{I}_n) = \mathbf{PQP} + \mathbf{P} = (\mathbf{PQ} + \mathbf{I}_m)\mathbf{P}$.
- Assuming the inverses exist, we can multiply by $(\mathbf{PQ} + \mathbf{I}_m)^{-1}$ on the left and $(\mathbf{QP} + \mathbf{I}_n)^{-1}$ on the right:

$$(\mathbf{PQ} + \mathbf{I}_m)^{-1}\mathbf{P}(\mathbf{QP} + \mathbf{I}_n)(\mathbf{QP} + \mathbf{I}_n)^{-1} = (\mathbf{PQ} + \mathbf{I}_m)^{-1}(\mathbf{PQ} + \mathbf{I}_m)\mathbf{P}(\mathbf{QP} + \mathbf{I}_n)^{-1}$$
$$(\mathbf{PQ} + \mathbf{I}_m)^{-1}\mathbf{P} = \mathbf{P}(\mathbf{QP} + \mathbf{I}_n)^{-1}$$

Proof of Alternative Formulation

- This relies on a common matrix identity (sometimes called the “push-through” identity).
- Let $\mathbf{P} \in \mathbb{R}^{m \times n}$ and $\mathbf{Q} \in \mathbb{R}^{n \times m}$.
- We have: $\mathbf{P}(\mathbf{QP} + \mathbf{I}_n) = \mathbf{PQP} + \mathbf{P} = (\mathbf{PQ} + \mathbf{I}_m)\mathbf{P}$.
- Assuming the inverses exist, we can multiply by $(\mathbf{PQ} + \mathbf{I}_m)^{-1}$ on the left and $(\mathbf{QP} + \mathbf{I}_n)^{-1}$ on the right:

$$(\mathbf{PQ} + \mathbf{I}_m)^{-1}\mathbf{P}(\mathbf{QP} + \mathbf{I}_n)(\mathbf{QP} + \mathbf{I}_n)^{-1} = (\mathbf{PQ} + \mathbf{I}_m)^{-1}(\mathbf{PQ} + \mathbf{I}_m)\mathbf{P}(\mathbf{QP} + \mathbf{I}_n)^{-1}$$
$$(\mathbf{PQ} + \mathbf{I}_m)^{-1}\mathbf{P} = \mathbf{P}(\mathbf{QP} + \mathbf{I}_n)^{-1}$$

- We get our result by setting $\mathbf{P} = \mathbf{X}^\top$ and $\mathbf{Q} = \frac{1}{\lambda N}\mathbf{X}$.

Proof of Alternative Formulation

- This relies on a common matrix identity (sometimes called the “push-through” identity).
- Let $\mathbf{P} \in \mathbb{R}^{m \times n}$ and $\mathbf{Q} \in \mathbb{R}^{n \times m}$.
- We have: $\mathbf{P}(\mathbf{Q}\mathbf{P} + \mathbf{I}_n) = \mathbf{P}\mathbf{Q}\mathbf{P} + \mathbf{P} = (\mathbf{P}\mathbf{Q} + \mathbf{I}_m)\mathbf{P}$.
- Assuming the inverses exist, we can multiply by $(\mathbf{P}\mathbf{Q} + \mathbf{I}_m)^{-1}$ on the left and $(\mathbf{Q}\mathbf{P} + \mathbf{I}_n)^{-1}$ on the right:

$$(\mathbf{P}\mathbf{Q} + \mathbf{I}_m)^{-1}\mathbf{P}(\mathbf{Q}\mathbf{P} + \mathbf{I}_n)(\mathbf{Q}\mathbf{P} + \mathbf{I}_n)^{-1} = (\mathbf{P}\mathbf{Q} + \mathbf{I}_m)^{-1}(\mathbf{P}\mathbf{Q} + \mathbf{I}_m)\mathbf{P}(\mathbf{Q}\mathbf{P} + \mathbf{I}_n)^{-1}$$
$$(\mathbf{P}\mathbf{Q} + \mathbf{I}_m)^{-1}\mathbf{P} = \mathbf{P}(\mathbf{Q}\mathbf{P} + \mathbf{I}_n)^{-1}$$

- We get our result by setting $\mathbf{P} = \mathbf{X}^\top$ and $\mathbf{Q} = \frac{1}{\lambda N}\mathbf{X}$.
- This switches a $D \times D$ inverse for an $N \times N$ inverse.

The Representer Theorem

The alternative form $\mathbf{w}^\star = \mathbf{X}^\top \boldsymbol{\alpha}^\star$ is not a coincidence.

Define $\boldsymbol{\alpha}^\star = \frac{1}{N}(\frac{1}{N}\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_N)^{-1}\mathbf{y}$. Then $\mathbf{w}^\star = \mathbf{X}^\top \boldsymbol{\alpha}^\star$.

The Representer Theorem

The alternative form $\mathbf{w}^\star = \mathbf{X}^\top \boldsymbol{\alpha}^\star$ is not a coincidence.

$$\text{Define } \boldsymbol{\alpha}^\star = \tfrac{1}{N}(\tfrac{1}{N}\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_N)^{-1}\mathbf{y}. \quad \text{Then } \mathbf{w}^\star = \mathbf{X}^\top \boldsymbol{\alpha}^\star.$$

This shows $\mathbf{w}^\star$ lies in the space spanned by the feature vectors (the rows of $\mathbf{X}$).

The Representer Theorem

The alternative form $\mathbf{w}^\star = \mathbf{X}^\top \boldsymbol{\alpha}^\star$ is not a coincidence.

$$\text{Define } \boldsymbol{\alpha}^\star = \tfrac{1}{N}(\tfrac{1}{N}\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_N)^{-1}\mathbf{y}. \quad \text{Then } \mathbf{w}^\star = \mathbf{X}^\top \boldsymbol{\alpha}^\star.$$

This shows $\mathbf{w}^\star$ lies in the space spanned by the feature vectors (the rows of $\mathbf{X}$).

The Representer Theorem (General) For any loss function $\mathcal{L}_n$, the solution $\mathbf{w}^\star$ to:

$$\min_{\mathbf{w}} \ \tfrac{1}{N}\sum_{n=1}^{N} \mathcal{L}_n(\mathbf{x}_n^\top \mathbf{w}, y_n) + \tfrac{\lambda}{2}\|\mathbf{w}\|^2$$

can be written as a linear combination of the data points:

$$\mathbf{w}^\star = \mathbf{X}^\top \boldsymbol{\alpha}^\star = \sum_{n=1}^{N} \alpha_n^\star \mathbf{x}_n$$

The Representer Theorem

The alternative form $\mathbf{w}^\star = \mathbf{X}^\top \boldsymbol{\alpha}^\star$ is not a coincidence.

$$\text{Define } \boldsymbol{\alpha}^\star = \tfrac{1}{N}(\tfrac{1}{N}\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_N)^{-1}\mathbf{y}. \quad \text{Then } \mathbf{w}^\star = \mathbf{X}^\top \boldsymbol{\alpha}^\star.$$

This shows $\mathbf{w}^\star$ lies in the space spanned by the feature vectors (the rows of $\mathbf{X}$).

The Representer Theorem (General) For any loss function $\mathcal{L}_n$, the solution $\mathbf{w}^\star$ to:

$$\min_{\mathbf{w}} \ \tfrac{1}{N}\sum_{n=1}^{N} \mathcal{L}_n(\mathbf{x}_n^\top \mathbf{w}, y_n) + \tfrac{\lambda}{2}\|\mathbf{w}\|^2$$

can be written as a linear combination of the data points:

$$\mathbf{w}^\star = \mathbf{X}^\top \boldsymbol{\alpha}^\star = \sum_{n=1}^{N} \alpha_n^\star \mathbf{x}_n$$

Proof (Intuition): Decompose $\mathbf{w} = \mathbf{w}_\parallel + \mathbf{w}_\perp$, where $\mathbf{w}_\parallel$ is in the span of $\mathbf{X}$ and $\mathbf{w}_\perp$ is orthogonal to it.

- The loss term $\mathcal{L}_n$ only depends on $\mathbf{x}_n^\top \mathbf{w} = \mathbf{x}_n^\top \mathbf{w}_\parallel$.
- The regularization term $\|\mathbf{w}\|^2 = \left\|\mathbf{w}_\parallel\right\|^2 + \|\mathbf{w}_\perp\|^2$.
- To minimize the cost, we must set $\mathbf{w}_\perp = 0$. Thus, $\mathbf{w}^\star = \mathbf{w}_\parallel^\star \in \text{span}(\mathbf{X})$.

Representer Theorem (Formal Proof)

- Let $\mathbf{w}^*$ be any optimal solution.

Representer Theorem (Formal Proof)

- Let $\mathbf{w}^*$ be any optimal solution.
- We can always decompose $\mathbf{w}^*$ into two orthogonal components:

$$\mathbf{w}^* = \underbrace{\sum_{n=1}^{N} \alpha_n \mathbf{x}_n}_{\mathbf{w}_\parallel \in \text{span}(\mathbf{X})} + \underbrace{\mathbf{u}}_{\mathbf{w}_\perp \perp \text{span}(\mathbf{X})}$$

where $\mathbf{u}^\top \mathbf{x}_n = 0$ for all n.

Representer Theorem (Formal Proof)

- Let $\mathbf{w}^*$ be any optimal solution.
- We can always decompose $\mathbf{w}^*$ into two orthogonal components:

$$\mathbf{w}^* = \underbrace{\sum_{n=1}^{N} \alpha_n \mathbf{x}_n}_{\mathbf{w}_\parallel \in \text{span}(\mathbf{X})} + \underbrace{\mathbf{u}}_{\mathbf{w}_\perp \perp \text{span}(\mathbf{X})}$$

  where $\mathbf{u}^\top \mathbf{x}_n = 0$ for all n.
- **Part 1: The Loss Term.** The loss only depends on $\mathbf{w}_\parallel$:

$$\mathbf{x}_n^\top \mathbf{w}^* = \mathbf{x}_n^\top (\mathbf{w}_\parallel + \mathbf{u}) = \mathbf{x}_n^\top \mathbf{w}_\parallel + \mathbf{x}_n^\top \mathbf{u} = \mathbf{x}_n^\top \mathbf{w}_\parallel$$

  So, $l(\mathbf{x}_n^\top \mathbf{w}^*, y_n) = l(\mathbf{x}_n^\top \mathbf{w}_\parallel, y_n)$.

Representer Theorem (Formal Proof)

- Let $\mathbf{w}^*$ be any optimal solution.
- We can always decompose $\mathbf{w}^*$ into two orthogonal components:

$$\mathbf{w}^* = \underbrace{\sum_{n=1}^{N} \alpha_n \mathbf{x}_n}_{\mathbf{w}_\parallel \in \text{span}(\mathbf{X})} + \underbrace{\mathbf{u}}_{\mathbf{w}_\perp \perp \text{span}(\mathbf{X})}$$

  where $\mathbf{u}^\top \mathbf{x}_n = 0$ for all n.
- **Part 1: The Loss Term.** The loss only depends on $\mathbf{w}_\parallel$:

$$\mathbf{x}_n^\top \mathbf{w}^* = \mathbf{x}_n^\top (\mathbf{w}_\parallel + \mathbf{u}) = \mathbf{x}_n^\top \mathbf{w}_\parallel + \mathbf{x}_n^\top \mathbf{u} = \mathbf{x}_n^\top \mathbf{w}_\parallel$$

  So, $l(\mathbf{x}_n^\top \mathbf{w}^*, y_n) = l(\mathbf{x}_n^\top \mathbf{w}_\parallel, y_n)$.
- **Part 2: The Regularization Term.** By Pythagorean theorem (since $\mathbf{w}_\parallel \perp \mathbf{u}$):

$$\|\mathbf{w}^*\|^2 = \|\mathbf{w}_\parallel + \mathbf{u}\|^2 = \|\mathbf{w}_\parallel\|^2 + \|\mathbf{u}\|^2$$

  This means $\|\mathbf{w}_\parallel\|^2 \leq \|\mathbf{w}^*\|^2$.

Representer Theorem (Formal Proof)

- Let $\mathbf{w}^*$ be any optimal solution.
- We can always decompose $\mathbf{w}^*$ into two orthogonal components:

$$\mathbf{w}^* = \underbrace{\sum_{n=1}^{N} \alpha_n \mathbf{x}_n}_{\mathbf{w}_\parallel \in \text{span}(\mathbf{X})} + \underbrace{\mathbf{u}}_{\mathbf{w}_\perp \perp \text{span}(\mathbf{X})}$$

  where $\mathbf{u}^\top \mathbf{x}_n = 0$ for all n.
- **Part 1: The Loss Term.** The loss only depends on $\mathbf{w}_\parallel$:

$$\mathbf{x}_n^\top \mathbf{w}^* = \mathbf{x}_n^\top (\mathbf{w}_\parallel + \mathbf{u}) = \mathbf{x}_n^\top \mathbf{w}_\parallel + \mathbf{x}_n^\top \mathbf{u} = \mathbf{x}_n^\top \mathbf{w}_\parallel$$

  So, $l(\mathbf{x}_n^\top \mathbf{w}^*, y_n) = l(\mathbf{x}_n^\top \mathbf{w}_\parallel, y_n)$.
- **Part 2: The Regularization Term.** By Pythagorean theorem (since $\mathbf{w}_\parallel \perp \mathbf{u}$):

$$\|\mathbf{w}^*\|^2 = \|\mathbf{w}_\parallel + \mathbf{u}\|^2 = \|\mathbf{w}_\parallel\|^2 + \|\mathbf{u}\|^2$$

  This means $\|\mathbf{w}_\parallel\|^2 \leq \|\mathbf{w}^*\|^2$.
- **Conclusion:** The total cost for $\mathbf{w}_\parallel$ is always less than or equal to the cost for $\mathbf{w}^*$.

Kernelized Ridge Regression

- The Representer Theorem lets us write an equivalent optimization problem in terms of $\boldsymbol{\alpha} \in \mathbb{R}^N$ instead of $\mathbf{w} \in \mathbb{R}^D$.

Kernelized Ridge Regression

- The Representer Theorem lets us write an equivalent optimization problem in terms of $\boldsymbol{\alpha} \in \mathbb{R}^N$ instead of $\mathbf{w} \in \mathbb{R}^D$.
- **Primal Problem (in $\mathbf{w}$):**

$$\mathbf{w}^\star = \arg\min_{\mathbf{w}} \ \tfrac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 \ + \ \tfrac{\lambda}{2} \|\mathbf{w}\|^2$$

Kernelized Ridge Regression

- The Representer Theorem lets us write an equivalent optimization problem in terms of $\boldsymbol{\alpha} \in \mathbb{R}^N$ instead of $\mathbf{w} \in \mathbb{R}^D$.
- **Primal Problem (in $\mathbf{w}$):**

$$\mathbf{w}^\star = \arg\min_{\mathbf{w}} \ \tfrac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 \ + \ \tfrac{\lambda}{2} \|\mathbf{w}\|^2$$

- **Dual Problem (in $\boldsymbol{\alpha}$):**

$$\boldsymbol{\alpha}^\star = \arg\min_{\boldsymbol{\alpha}} \ \frac{1}{2}\boldsymbol{\alpha}^\top (\tfrac{1}{N}\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_N)\boldsymbol{\alpha} - \tfrac{1}{N}\boldsymbol{\alpha}^\top \mathbf{y}$$

Kernelized Ridge Regression

- The Representer Theorem lets us write an equivalent optimization problem in terms of $\boldsymbol{\alpha} \in \mathbb{R}^N$ instead of $\mathbf{w} \in \mathbb{R}^D$.
- **Primal Problem (in $\mathbf{w}$):**

$$\mathbf{w}^\star = \arg\min_{\mathbf{w}} \ \tfrac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 \ + \ \tfrac{\lambda}{2} \|\mathbf{w}\|^2$$

- **Dual Problem (in $\boldsymbol{\alpha}$):**

$$\boldsymbol{\alpha}^\star = \arg\min_{\boldsymbol{\alpha}} \ \frac{1}{2}\boldsymbol{\alpha}^\top (\tfrac{1}{N}\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_N)\boldsymbol{\alpha} - \tfrac{1}{N}\boldsymbol{\alpha}^\top \mathbf{y}$$

- The dual problem only depends on the data through the kernel matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.

The Kernel Matrix

- The dual formulations for SVM and Ridge Regression depend only on the $N \times N$ matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.

The Kernel Matrix

- The dual formulations for SVM and Ridge Regression depend only on the $N \times N$ matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.
- This is called the Kernel Matrix or Gram Matrix.

The Kernel Matrix

- The dual formulations for SVM and Ridge Regression depend only on the $N \times N$ matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.
- This is called the Kernel Matrix or Gram Matrix.
- Each element $\mathbf{K}_{ij}$ is the inner product between two data points:

$$\mathbf{K}_{ij} = \mathbf{x}_i^\top \mathbf{x}_j$$

The Kernel Matrix

- The dual formulations for SVM and Ridge Regression depend only on the $N \times N$ matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.
- This is called the Kernel Matrix or Gram Matrix.
- Each element $\mathbf{K}_{ij}$ is the inner product between two data points:

$$\mathbf{K}_{ij} = \mathbf{x}_i^\top \mathbf{x}_j$$

- $$\mathbf{K} = \mathbf{X}\mathbf{X}^\top = \begin{pmatrix} \mathbf{x}_1^\top \mathbf{x}_1 & \mathbf{x}_1^\top \mathbf{x}_2 & \cdots & \mathbf{x}_1^\top \mathbf{x}_N \\ \mathbf{x}_2^\top \mathbf{x}_1 & \mathbf{x}_2^\top \mathbf{x}_2 & \cdots & \mathbf{x}_2^\top \mathbf{x}_N \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_N^\top \mathbf{x}_1 & \mathbf{x}_N^\top \mathbf{x}_2 & \cdots & \mathbf{x}_N^\top \mathbf{x}_N \end{pmatrix}$$

The Kernel Matrix

- The dual formulations for SVM and Ridge Regression depend only on the $N \times N$ matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.
- This is called the Kernel Matrix or Gram Matrix.
- Each element $\mathbf{K}_{ij}$ is the inner product between two data points:

$$\mathbf{K}_{ij} = \mathbf{x}_i^\top \mathbf{x}_j$$

- $\mathbf{K} = \mathbf{X}\mathbf{X}^\top = \begin{pmatrix} \mathbf{x}_1^\top \mathbf{x}_1 & \mathbf{x}_1^\top \mathbf{x}_2 & \cdots & \mathbf{x}_1^\top \mathbf{x}_N \\ \mathbf{x}_2^\top \mathbf{x}_1 & \mathbf{x}_2^\top \mathbf{x}_2 & \cdots & \mathbf{x}_2^\top \mathbf{x}_N \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_N^\top \mathbf{x}_1 & \mathbf{x}_N^\top \mathbf{x}_2 & \cdots & \mathbf{x}_N^\top \mathbf{x}_N \end{pmatrix}$
- This matrix is symmetric ($\mathbf{K} = \mathbf{K}^\top$) and positive semi-definite ($\mathbf{K} \geq 0$).

The Kernel Trick: Feature Maps

- What if our features $\mathbf{x}$ aren't good enough? We can augment them with a feature map $\phi(\mathbf{x})$.

The Kernel Trick: Feature Maps

- What if our features $\mathbf{x}$ aren't good enough? We can augment them with a feature map $\phi(\mathbf{x})$.
- Example: $\mathbf{x} = [x_1, x_2] \rightarrow \phi(\mathbf{x}) = [x_1^2, x_2^2, \sqrt{2}x_1x_2]$

The Kernel Trick: Feature Maps

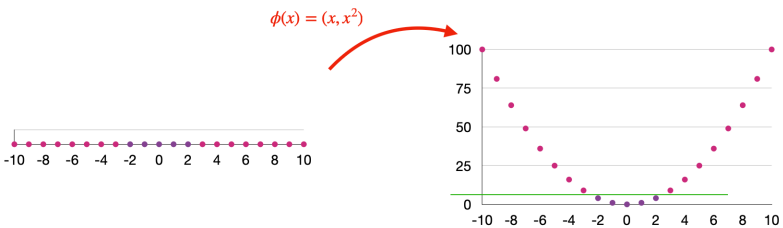
- What if our features $\mathbf{x}$ aren't good enough? We can augment them with a feature map $\boldsymbol{\phi}(\mathbf{x})$.
- Example: $\mathbf{x} = [x_1, x_2] \to \boldsymbol{\phi}(\mathbf{x}) = [x_1^2, x_2^2, \sqrt{2}x_1x_2]$
- Our algorithms now run in this new, higher-dimensional feature space.
  - $\mathbf{w}$ is now in this new space.
  - The kernel matrix becomes $\mathbf{K}_{ij} = \boldsymbol{\phi}(\mathbf{x}_i)^\top \boldsymbol{\phi}(\mathbf{x}_j)$.

The Kernel Trick: Feature Maps

- What if our features $\mathbf{x}$ aren't good enough? We can augment them with a feature map $\phi(\mathbf{x})$.
- Example: $\mathbf{x} = [x_1, x_2] \to \phi(\mathbf{x}) = [x_1^2, x_2^2, \sqrt{2}x_1x_2]$
- Our algorithms now run in this new, higher-dimensional feature space.
  - $\mathbf{w}$ is now in this new space.
  - The kernel matrix becomes $\mathbf{K}_{ij} = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$.
- **Problem:** If $\phi(\mathbf{x})$ is very high-dimensional (or infinite!), computing $\phi(\mathbf{x})$ for all data and then $\mathbf{K}$ is extremely expensive or impossible.

The Kernel Trick: Feature Maps

- What if our features $\mathbf{x}$ aren't good enough? We can augment them with a feature map $\phi(\mathbf{x})$.
- Example: $\mathbf{x} = [x_1, x_2] \to \phi(\mathbf{x}) = [x_1^2, x_2^2, \sqrt{2}x_1x_2]$
- Our algorithms now run in this new, higher-dimensional feature space.
  - $\mathbf{w}$ is now in this new space.
  - The kernel matrix becomes $\mathbf{K}_{ij} = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$.
- **Problem:** If $\phi(\mathbf{x})$ is very high-dimensional (or infinite!), computing $\phi(\mathbf{x})$ for all data and then $\mathbf{K}$ is extremely expensive or impossible.
- **Example:** Data not linearly separable in 1D.



Kernel Matrix with Feature Maps

- When we use a feature map $\phi : \mathbb{R}^D \to \mathbb{R}^{\tilde{D}}$, our data matrix $\mathbf{X}$ becomes $\mathbf{\Phi}$.

Kernel Matrix with Feature Maps

- When we use a feature map $\phi : \mathbb{R}^D \to \mathbb{R}^{\tilde{D}}$, our data matrix $\mathbf{X}$ becomes $\mathbf{\Phi}$.
- The kernel matrix is now computed in the new feature space:

$$\mathbf{K}_{ij} = \boldsymbol{\phi}(\mathbf{x}_i)^\top \boldsymbol{\phi}(\mathbf{x}_j)$$

Kernel Matrix with Feature Maps

- When we use a feature map $\phi : \mathbb{R}^D \to \mathbb{R}^{\tilde{D}}$, our data matrix $\mathbf{X}$ becomes $\mathbf{\Phi}$.
- The kernel matrix is now computed in the new feature space:

$$\mathbf{K}_{ij} = \boldsymbol{\phi}(\mathbf{x}_i)^\top \boldsymbol{\phi}(\mathbf{x}_j)$$

- This is $\mathbf{K} = \mathbf{\Phi}\mathbf{\Phi}^\top$.

Kernel Matrix with Feature Maps

- When we use a feature map $\phi : \mathbb{R}^D \to \mathbb{R}^{\tilde{D}}$, our data matrix $\mathbf{X}$ becomes $\mathbf{\Phi}$.
- The kernel matrix is now computed in the new feature space:

$$\mathbf{K}_{ij} = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$$

- This is $\mathbf{K} = \mathbf{\Phi}\mathbf{\Phi}^\top$.
- **The Problem:** If the new dimension $\tilde{D}$ is very large (e.g., $D \ll \tilde{D}$), computing *one* inner product $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ costs $O(\tilde{D})$.

Kernel Matrix with Feature Maps

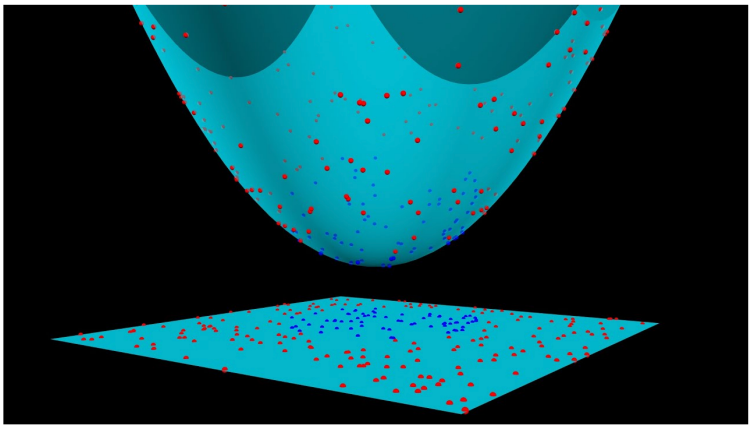
- When we use a feature map $\phi : \mathbb{R}^D \to \mathbb{R}^{\tilde{D}}$, our data matrix $\mathbf{X}$ becomes $\mathbf{\Phi}$.
- The kernel matrix is now computed in the new feature space:

$$\mathbf{K}_{ij} = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$$

- This is $\mathbf{K} = \mathbf{\Phi}\mathbf{\Phi}^\top$.
- **The Problem:** If the new dimension $\tilde{D}$ is very large (e.g., $D \ll \tilde{D}$), computing *one* inner product $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ costs $O(\tilde{D})$.
- This is too expensive, or impossible if $\tilde{D} = \infty$.

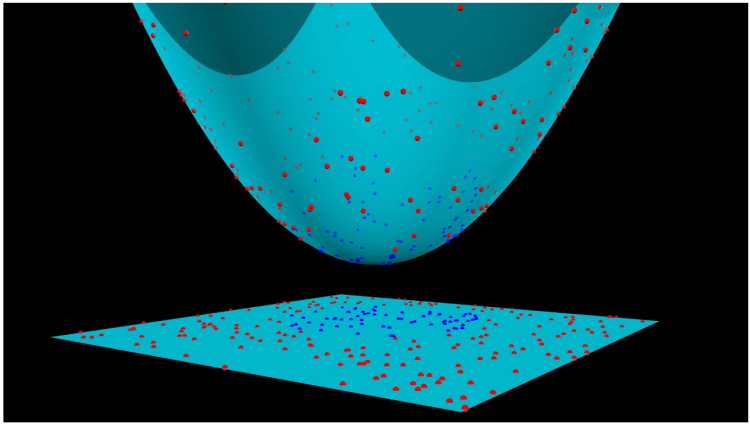
Usefulness of Feature Spaces

Data that is not linearly separable in 2D...



Usefulness of Feature Spaces

Data that is not linearly separable in 2D...



...can become linearly separable when mapped to a higher dimension (3D).

The Kernel Trick: The “Shortcut”

The Kernel Trick: If we can find a kernel function $\kappa(\mathbf{x}, \mathbf{x}')$ that computes the inner product in the feature space *directly*, we never need to build $\boldsymbol{\phi}(\mathbf{x})$.

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\phi}(\mathbf{x}_i)^\top \boldsymbol{\phi}(\mathbf{x}_j)$$

The Kernel Trick: The “Shortcut”

The Kernel Trick: If we can find a kernel function $\kappa(\mathbf{x}, \mathbf{x}')$ that computes the inner product in the feature space *directly*, we never need to build $\boldsymbol{\phi}(\mathbf{x})$.

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\phi}(\mathbf{x}_i)^\top \boldsymbol{\phi}(\mathbf{x}_j)$$

- We only need to compute the $N \times N$ kernel matrix $\mathbf{K}_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$.
- All “kernelized” algorithms (like SVM dual, Ridge dual) only need $\mathbf{K}$.
- We can work in an incredibly high-dimensional space, as long as we have an efficient $\kappa(\cdot, \cdot)$.

Example 1: Polynomial Kernel

- **Kernel Function:** $\kappa(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}')^2$

Example 1: Polynomial Kernel

- **Kernel Function:** $\kappa(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}')^2$
- Let $\mathbf{x} \in \mathbb{R}^3$. $\kappa(\mathbf{x}, \mathbf{x}') = (x_1 x_1' + x_2 x_2' + x_3 x_3')^2$

Example 1: Polynomial Kernel

- **Kernel Function:** $\kappa(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}')^2$
- Let $\mathbf{x} \in \mathbb{R}^3$. $\kappa(\mathbf{x}, \mathbf{x}') = (x_1 x_1' + x_2 x_2' + x_3 x_3')^2$
- **Implicit Feature Map $\phi(\mathbf{x}) \in \mathbb{R}^6$:**

$$\phi(\mathbf{x})^\top = \left[x_1^2, x_2^2, x_3^2, \sqrt{2}x_1x_2, \sqrt{2}x_1x_3, \sqrt{2}x_2x_3\right]$$

Example 1: Polynomial Kernel

- **Kernel Function:** $\kappa(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}')^2$
- Let $\mathbf{x} \in \mathbb{R}^3$. $\kappa(\mathbf{x}, \mathbf{x}') = (x_1 x_1' + x_2 x_2' + x_3 x_3')^2$
- **Implicit Feature Map $\phi(\mathbf{x}) \in \mathbb{R}^6$:**

$$\phi(\mathbf{x})^\top = \left[x_1^2, x_2^2, x_3^2, \sqrt{2}x_1x_2, \sqrt{2}x_1x_3, \sqrt{2}x_2x_3\right]$$

- **Cost:**
  - Computing $\kappa(\mathbf{x}, \mathbf{x}')$: 1 inner product ($O(D)$), 1 multiplication.
  - Computing $\phi(\mathbf{x})$ and then the dot product: $O(D^2)$ operations.

Example 1: Polynomial Kernel

- **Kernel Function:** $\kappa(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}')^2$
- Let $\mathbf{x} \in \mathbb{R}^3$. $\kappa(\mathbf{x}, \mathbf{x}') = (x_1 x_1' + x_2 x_2' + x_3 x_3')^2$
- **Implicit Feature Map $\phi(\mathbf{x}) \in \mathbb{R}^6$:**

$$\phi(\mathbf{x})^\top = \left[x_1^2, x_2^2, x_3^2, \sqrt{2}x_1x_2, \sqrt{2}x_1x_3, \sqrt{2}x_2x_3\right]$$

- **Cost:**
  - Computing $\kappa(\mathbf{x}, \mathbf{x}')$: 1 inner product ($O(D)$), 1 multiplication.
  - Computing $\phi(\mathbf{x})$ and then the dot product: $O(D^2)$ operations.
- The kernel function is a huge computational shortcut!

Example 2: RBF (Gaussian) Kernel

- **Kernel Function:**

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \left\|\mathbf{x} - \mathbf{x}'\right\|^2\right)$$

Example 2: RBF (Gaussian) Kernel

- **Kernel Function:**

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \left\|\mathbf{x} - \mathbf{x}'\right\|^2\right)$$

- This is a very popular and powerful kernel. (Note: $\gamma = 1$ in your notes, e.g., $\exp(-(\mathbf{x} - \mathbf{x}')^\top(\mathbf{x} - \mathbf{x}'))$)

Example 2: RBF (Gaussian) Kernel

- **Kernel Function:**

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \left\|\mathbf{x} - \mathbf{x}'\right\|^2\right)$$

- This is a very popular and powerful kernel. (Note: $\gamma = 1$ in your notes, e.g., $\exp(-(\mathbf{x} - \mathbf{x}')^\top(\mathbf{x} - \mathbf{x}'))$)
- **Implicit Feature Map $\phi(\mathbf{x})$:**
  - The corresponding feature space is **infinite-dimensional**!

Example 2: RBF (Gaussian) Kernel

- **Kernel Function:**

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \left\|\mathbf{x} - \mathbf{x}'\right\|^2\right)$$

- This is a very popular and powerful kernel. (Note: $\gamma = 1$ in your notes, e.g., $\exp(-(\mathbf{x} - \mathbf{x}')^\top(\mathbf{x} - \mathbf{x}'))$)
- **Implicit Feature Map $\phi(\mathbf{x})$:**
  - The corresponding feature space is **infinite-dimensional**!
- We can see this by expanding the simple 1D case ($x \in \mathbb{R}$, $\gamma = 1$):

$$\begin{aligned}\kappa(x, x') &= e^{-(x-x')^2} = e^{-x^2} e^{-(x')^2} e^{2xx'} \\ &= e^{-x^2} e^{-(x')^2} \sum_{k=0}^{\infty} \frac{2^k (x)^k (x')^k}{k!}\end{aligned}$$

Example 2: RBF (Gaussian) Kernel

- **Kernel Function:**

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \left\|\mathbf{x} - \mathbf{x}'\right\|^2\right)$$

- This is a very popular and powerful kernel. (Note: $\gamma = 1$ in your notes, e.g., $\exp(-(\mathbf{x} - \mathbf{x}')^\top(\mathbf{x} - \mathbf{x}'))$)
- **Implicit Feature Map $\phi(\mathbf{x})$:**
  - The corresponding feature space is **infinite-dimensional**!
- We can see this by expanding the simple 1D case ($x \in \mathbb{R}$, $\gamma = 1$):

$$\begin{aligned}\kappa(x, x') &= e^{-(x-x')^2} = e^{-x^2} e^{-(x')^2} e^{2xx'} \\ &= e^{-x^2} e^{-(x')^2} \sum_{k=0}^{\infty} \frac{2^k (x)^k (x')^k}{k!}\end{aligned}$$

- This is an inner product between two infinite vectors:

$$\phi(x)^\top = e^{-x^2}\left[\ldots, \sqrt{\frac{2^k}{k!}} x^k, \ldots\right]$$

Example 2: RBF (Gaussian) Kernel

- **Kernel Function:**

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \left\|\mathbf{x} - \mathbf{x}'\right\|^2\right)$$

- This is a very popular and powerful kernel. (Note: $\gamma = 1$ in your notes, e.g., $\exp(-(\mathbf{x} - \mathbf{x}')^\top(\mathbf{x} - \mathbf{x}'))$)
- **Implicit Feature Map $\phi(\mathbf{x})$:**
  - The corresponding feature space is **infinite-dimensional**!
- We can see this by expanding the simple 1D case ($x \in \mathbb{R}$, $\gamma = 1$):

$$\begin{aligned}\kappa(x, x') &= e^{-(x-x')^2} = e^{-x^2} e^{-(x')^2} e^{2xx'} \\ &= e^{-x^2} e^{-(x')^2} \sum_{k=0}^{\infty} \frac{2^k (x)^k (x')^k}{k!}\end{aligned}$$

- This is an inner product between two infinite vectors:

$$\phi(x)^\top = e^{-x^2}\left[\ldots, \sqrt{\frac{2^k}{k!}} x^k, \ldots\right]$$

- The Kernel Trick lets us compute this impossible-to-build inner product

Building New Kernels from Old

If κ_1 and κ_2 are valid kernels, so are:

- **Positive Scaling:** $\kappa(\mathbf{x}, \mathbf{x}') = c\,\kappa_1(\mathbf{x}, \mathbf{x}')$ for $c \geq 0$.

Building New Kernels from Old

If κ_1 and κ_2 are valid kernels, so are:

- **Positive Scaling:** $\kappa(\mathbf{x}, \mathbf{x}') = c\,\kappa_1(\mathbf{x}, \mathbf{x}')$ for $c \geq 0$.
- **Sum:** $\kappa(\mathbf{x}, \mathbf{x}') = \kappa_1(\mathbf{x}, \mathbf{x}') + \kappa_2(\mathbf{x}, \mathbf{x}')$.
  - Proof: $\boldsymbol{\phi}(\cdot) = [\boldsymbol{\phi}_1(\cdot), \boldsymbol{\phi}_2(\cdot)]$

Building New Kernels from Old

If κ_1 and κ_2 are valid kernels, so are:

- **Positive Scaling:** $\kappa(\mathbf{x}, \mathbf{x}') = c\,\kappa_1(\mathbf{x}, \mathbf{x}')$ for $c \geq 0$.
- **Sum:** $\kappa(\mathbf{x}, \mathbf{x}') = \kappa_1(\mathbf{x}, \mathbf{x}') + \kappa_2(\mathbf{x}, \mathbf{x}')$.
  - Proof: $\phi(\cdot) = [\phi_1(\cdot), \phi_2(\cdot)]$
- **Product:** $\kappa(\mathbf{x}, \mathbf{x}') = \kappa_1(\mathbf{x}, \mathbf{x}')\kappa_2(\mathbf{x}, \mathbf{x}')$.
  - Proof: $\phi(\cdot)$ has entries $(\phi_1(\cdot))_i(\phi_2(\cdot))_j$

Building New Kernels from Old

If κ_1 and κ_2 are valid kernels, so are:

- **Positive Scaling:** $\kappa(\mathbf{x}, \mathbf{x}') = c\,\kappa_1(\mathbf{x}, \mathbf{x}')$ for $c \geq 0$.
- **Sum:** $\kappa(\mathbf{x}, \mathbf{x}') = \kappa_1(\mathbf{x}, \mathbf{x}') + \kappa_2(\mathbf{x}, \mathbf{x}')$.
  - Proof: $\phi(\cdot) = [\phi_1(\cdot), \phi_2(\cdot)]$
- **Product:** $\kappa(\mathbf{x}, \mathbf{x}') = \kappa_1(\mathbf{x}, \mathbf{x}')\kappa_2(\mathbf{x}, \mathbf{x}')$.
  - Proof: $\phi(\cdot)$ has entries $(\phi_1(\cdot))_i(\phi_2(\cdot))_j$
- **Function mapping:** $\kappa(\mathbf{x}, \mathbf{x}') = f(\mathbf{x})f(\mathbf{x}')$ for any real-valued f.

Proofs for Building Kernels

- **Proof (Sum):** $\kappa(\mathbf{x}, \mathbf{x}') = \alpha\kappa_1(\mathbf{x}, \mathbf{x}') + \beta\kappa_2(\mathbf{x}, \mathbf{x}')$
  - We can define a new feature map $\boldsymbol{\phi}(\mathbf{x})$ by concatenating the scaled original feature maps:

$$\boldsymbol{\phi}(\mathbf{x}) = \begin{bmatrix} \sqrt{\alpha}\boldsymbol{\phi}_1(\mathbf{x}) \\ \sqrt{\beta}\boldsymbol{\phi}_2(\mathbf{x}) \end{bmatrix} \in \mathbb{R}^{D_1+D_2}$$

  - The inner product is then:

$$\boldsymbol{\phi}(\mathbf{x})^\top \boldsymbol{\phi}(\mathbf{x}') = \alpha\boldsymbol{\phi}_1(\mathbf{x})^\top \boldsymbol{\phi}_1(\mathbf{x}') + \beta\boldsymbol{\phi}_2(\mathbf{x})^\top \boldsymbol{\phi}_2(\mathbf{x}')$$

Proofs for Building Kernels

- **Proof (Sum):** $\kappa(\mathbf{x}, \mathbf{x}') = \alpha\kappa_1(\mathbf{x}, \mathbf{x}') + \beta\kappa_2(\mathbf{x}, \mathbf{x}')$
  - We can define a new feature map $\boldsymbol{\phi}(\mathbf{x})$ by concatenating the scaled original feature maps:

$$\boldsymbol{\phi}(\mathbf{x}) = \begin{bmatrix} \sqrt{\alpha}\boldsymbol{\phi}_1(\mathbf{x}) \\ \sqrt{\beta}\boldsymbol{\phi}_2(\mathbf{x}) \end{bmatrix} \in \mathbb{R}^{D_1+D_2}$$

  - The inner product is then:

$$\boldsymbol{\phi}(\mathbf{x})^\top \boldsymbol{\phi}(\mathbf{x}') = \alpha\boldsymbol{\phi}_1(\mathbf{x})^\top \boldsymbol{\phi}_1(\mathbf{x}') + \beta\boldsymbol{\phi}_2(\mathbf{x})^\top \boldsymbol{\phi}_2(\mathbf{x}')$$

- **Proof (Product):** $\kappa(\mathbf{x}, \mathbf{x}') = \kappa_1(\mathbf{x}, \mathbf{x}')\kappa_2(\mathbf{x}, \mathbf{x}')$
  - We define a new feature map $\boldsymbol{\phi}(\mathbf{x})$ as the tensor (outer) product of the original features.
  - The new feature vector contains all possible products $(\boldsymbol{\phi}_1(\mathbf{x}))_i(\boldsymbol{\phi}_2(\mathbf{x}))_j$.
  - The inner product becomes:

$$\begin{aligned} \boldsymbol{\phi}(\mathbf{x})^\top \boldsymbol{\phi}(\mathbf{x}') &= \sum_{i,j} (\boldsymbol{\phi}_1(\mathbf{x}))_i (\boldsymbol{\phi}_2(\mathbf{x}))_j (\boldsymbol{\phi}_1(\mathbf{x}'))_i (\boldsymbol{\phi}_2(\mathbf{x}'))_j \\ &= \left(\sum_i (\boldsymbol{\phi}_1(\mathbf{x}))_i (\boldsymbol{\phi}_1(\mathbf{x}'))_i \right) \left(\sum_j (\boldsymbol{\phi}_2(\mathbf{x}))_j (\boldsymbol{\phi}_2(\mathbf{x}'))_j \right) \\ &= \kappa_1(\mathbf{x}, \mathbf{x}')\kappa_2(\mathbf{x}, \mathbf{x}') \end{aligned}$$

Prediction with Kernels

The representer theorem gives an expansion in training examples:

$$\mathbf{w}^\star = \sum_{n=1}^{N} c_n \boldsymbol{\phi}(\mathbf{x}_n).$$

We can evaluate the score without constructing the feature vectors:

$$f(\mathbf{x}) = \mathbf{w}^{\star\top} \boldsymbol{\phi}(\mathbf{x}) = \sum_{n=1}^{N} c_n \boldsymbol{\phi}(\mathbf{x}_n)^\top \boldsymbol{\phi}(\mathbf{x})$$

$$= \sum_{n=1}^{N} c_n \kappa(\mathbf{x}_n, \mathbf{x}), \qquad \hat{y} = \operatorname{sign}(f(\mathbf{x})).$$

For our SVM formulation $c_n = \alpha_n^\star y_n / (\lambda N)$. Only support vectors with $\alpha_n^\star > 0$ contribute.

Prediction needs learned coefficients and kernel evaluations, even when the feature space is infinite-dimensional.

Properties of Kernels: Mercer's Condition

How do we know if a function $\kappa(\mathbf{x}, \mathbf{x}')$ is a valid kernel?
(i.e., corresponds to an inner product in *some* feature space?)

Properties of Kernels: Mercer's Condition

How do we know if a function $\kappa(\mathbf{x}, \mathbf{x}')$ is a valid kernel?
(i.e., corresponds to an inner product in *some* feature space?)

Mercer's Condition: A function κ is a valid kernel if and only if:

Properties of Kernels: Mercer's Condition

How do we know if a function $\kappa(\mathbf{x}, \mathbf{x}')$ is a valid kernel?
(i.e., corresponds to an inner product in *some* feature space?)

Mercer's Condition: A function κ is a valid kernel if and only if:

1. It is **symmetric**:

$$\kappa(\mathbf{x}, \mathbf{x}') = \kappa(\mathbf{x}', \mathbf{x})$$

Properties of Kernels: Mercer's Condition

How do we know if a function $\kappa(\mathbf{x}, \mathbf{x}')$ is a valid kernel?
(i.e., corresponds to an inner product in *some* feature space?)

Mercer's Condition: A function κ is a valid kernel if and only if:

1. It is **symmetric**:

$$\kappa(\mathbf{x}, \mathbf{x}') = \kappa(\mathbf{x}', \mathbf{x})$$

2. The kernel matrix $\mathbf{K}$ is **positive semi-definite** for *any* set of points $\{\mathbf{x}_n\}_{n=1}^N$.

$$\mathbf{K}_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j) \Longrightarrow \mathbf{c}^\top \mathbf{K} \mathbf{c} \geq 0 \text{ for all } \mathbf{c} \in \mathbb{R}^N$$

Recap: Key Ideas

- Many algorithms (SVM, Ridge Regression, PCA) can be “kernelized” because their solutions can be written to depend only on inner products, $\mathbf{X}\mathbf{X}^\top$.

Recap: Key Ideas

- Many algorithms (SVM, Ridge Regression, PCA) can be "kernelized" because their solutions can be written to depend only on inner products, $\mathbf{X}\mathbf{X}^\top$.
- This allows us to generalize a linear model by replacing the simple inner product $\mathbf{x}^\top\mathbf{x}'$ with a non-linear kernel function $\kappa(\mathbf{x}, \mathbf{x}')$.

Recap: Key Ideas

- Many algorithms (SVM, Ridge Regression, PCA) can be "kernelized" because their solutions can be written to depend only on inner products, $\mathbf{X}\mathbf{X}^\top$.
- This allows us to generalize a linear model by replacing the simple inner product $\mathbf{x}^\top\mathbf{x}'$ with a non-linear kernel function $\kappa(\mathbf{x}, \mathbf{x}')$.
- This creates a non-linear model in the original input space, but one that is still linear (and convex) in a new, high-dimensional feature space.

Recap: Key Ideas

- Many algorithms (SVM, Ridge Regression, PCA) can be "kernelized" because their solutions can be written to depend only on inner products, $\mathbf{X}\mathbf{X}^\top$.
- This allows us to generalize a linear model by replacing the simple inner product $\mathbf{x}^\top\mathbf{x}'$ with a non-linear kernel function $\kappa(\mathbf{x}, \mathbf{x}')$.
- This creates a non-linear model in the original input space, but one that is still linear (and convex) in a new, high-dimensional feature space.
- The Kernel Trick is that we can do this *without ever computing* the feature maps $\phi(\mathbf{x})$, which can be very high-dim or even infinite-dim.

Recap: Key Ideas

- Many algorithms (SVM, Ridge Regression, PCA) can be "kernelized" because their solutions can be written to depend only on inner products, $\mathbf{X}\mathbf{X}^\top$.
- This allows us to generalize a linear model by replacing the simple inner product $\mathbf{x}^\top\mathbf{x}'$ with a non-linear kernel function $\kappa(\mathbf{x}, \mathbf{x}')$.
- This creates a non-linear model in the original input space, but one that is still linear (and convex) in a new, high-dimensional feature space.
- The Kernel Trick is that we can do this *without ever computing* the feature maps $\phi(\mathbf{x})$, which can be very high-dim or even infinite-dim.
- To make a new prediction, we just need to compute the kernel (similarity) between the new point $\mathbf{x}$ and all the training points $\mathbf{x}_n$.

A Brief History

- Support Vector Machines are a set of powerful supervised learning methods based on the concept of maximizing the margin.

A Brief History

- Support Vector Machines are a set of powerful supervised learning methods based on the concept of maximizing the margin.
- The idea was developed by **Vladimir Vapnik** and colleagues.

A Brief History

- Support Vector Machines are a set of powerful supervised learning methods based on the concept of maximizing the margin.
- The idea was developed by **Vladimir Vapnik** and colleagues.
- **Key Papers:**
  - **1992 (Linearly Separable):** Boser, Guyon, and Vapnik introduce the optimal margin classifier.
    - Solution is a linear combination of "supporting patterns" (Support Vectors).
  - **1995 (Non-Separable):** Cortes and Vapnik extend the idea to non-separable data (the "Soft Margin").
    - Introduces non-linear mapping to a high-dimension feature space.

Motivation: Reframing Classification Losses

- Let's reconsider binary classification with data $(\mathbf{x}_n, \tilde{y}_n)$, where $\tilde{y}_n \in \{0, 1\}$.

Motivation: Reframing Classification Losses

- Let's reconsider binary classification with data $(\mathbf{x}_n, \tilde{y}_n)$, where $\tilde{y}_n \in \{0, 1\}$.
- **Least Squares:** $\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} (\tilde{y}_n - \mathbf{x}_n^\top \mathbf{w})^2$

Motivation: Reframing Classification Losses

- Let's reconsider binary classification with data $(\mathbf{x}_n, \tilde{y}_n)$, where $\tilde{y}_n \in \{0, 1\}$.
- **Least Squares:** $\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} (\tilde{y}_n - \mathbf{x}_n^\top \mathbf{w})^2$
- **Logistic Regression:** $\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} \log(1 + e^{\mathbf{x}_n^\top \mathbf{w}}) - \tilde{y}_n \mathbf{x}_n^\top \mathbf{w}$

Motivation: Reframing Classification Losses

- Let's reconsider binary classification with data $(\mathbf{x}_n, \tilde{y}_n)$, where $\tilde{y}_n \in \{0, 1\}$.
- **Least Squares:** $\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} (\tilde{y}_n - \mathbf{x}_n^\top \mathbf{w})^2$
- **Logistic Regression:** $\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} \log(1 + e^{\mathbf{x}_n^\top \mathbf{w}}) - \tilde{y}_n \mathbf{x}_n^\top \mathbf{w}$
- It's often more convenient to use labels $y_n \in \{-1, 1\}$.

Motivation: Reframing Classification Losses

- Let's reconsider binary classification with data $(\mathbf{x}_n, \tilde{y}_n)$, where $\tilde{y}_n \in \{0, 1\}$.
- **Least Squares:** $\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} (\tilde{y}_n - \mathbf{x}_n^\top \mathbf{w})^2$
- **Logistic Regression:** $\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} \log(1 + e^{\mathbf{x}_n^\top \mathbf{w}}) - \tilde{y}_n \mathbf{x}_n^\top \mathbf{w}$
- It's often more convenient to use labels $y_n \in \{-1, 1\}$.
- We use the mapping: $y_n = 2\tilde{y}_n - 1 \leftrightarrow \tilde{y}_n = \frac{1}{2}(y_n + 1)$.

Motivation: Reframing Classification Losses

- Let's reconsider binary classification with data $(\mathbf{x}_n, \tilde{y}_n)$, where $\tilde{y}_n \in \{0, 1\}$.
- **Least Squares:** $\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} (\tilde{y}_n - \mathbf{x}_n^\top \mathbf{w})^2$
- **Logistic Regression:** $\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} \log(1 + e^{\mathbf{x}_n^\top \mathbf{w}}) - \tilde{y}_n \mathbf{x}_n^\top \mathbf{w}$
- It's often more convenient to use labels $y_n \in \{-1, 1\}$.
- We use the mapping: $y_n = 2\tilde{y}_n - 1 \leftrightarrow \tilde{y}_n = \frac{1}{2}(y_n + 1)$.
- Let's see how our loss functions change with this new y_n.

Motivation: Deriving the MSE Loss

- For Least Squares, let's assume bias e_0 is component 0. Define $\tilde{\mathbf{w}} = \frac{1}{2}(\mathbf{w} + e_0)$ (this is a one-to-one mapping).

Motivation: Deriving the MSE Loss

- For Least Squares, let's assume bias e_0 is component 0. Define $\tilde{\mathbf{w}} = \frac{1}{2}(\mathbf{w} + e_0)$ (this is a one-to-one mapping).
- The derivation becomes:

$$
\begin{aligned}
4\sum_{n=1}^{N}(\tilde{y}_n - \mathbf{x}_n^\top \tilde{\mathbf{w}})^2 &= 4\sum_{n=1}^{N}\left(\frac{1}{2}(y_n + 1) - \frac{1}{2}\mathbf{x}_n^\top(\mathbf{w} + e_0)\right)^2 \\
&= \sum_{n=1}^{N}((y_n + 1) - \mathbf{x}_n^\top(\mathbf{w} + e_0))^2 = \sum_{n=1}^{N}(y_n - \mathbf{x}_n^\top \mathbf{w})^2 \\
&\overset{(a)}{=} \sum_{n=1}^{N}(1 - y_n \mathbf{x}_n^\top \mathbf{w})^2
\end{aligned}
$$

Motivation: Deriving the MSE Loss

- For Least Squares, let's assume bias e_0 is component 0. Define $\tilde{\mathbf{w}} = \frac{1}{2}(\mathbf{w} + e_0)$ (this is a one-to-one mapping).
- The derivation becomes:

$$
\begin{aligned}
4\sum_{n=1}^{N}(\tilde{y}_n - \mathbf{x}_n^\top \tilde{\mathbf{w}})^2 &= 4\sum_{n=1}^{N}\left(\frac{1}{2}(y_n + 1) - \frac{1}{2}\mathbf{x}_n^\top(\mathbf{w} + e_0)\right)^2 \\
&= \sum_{n=1}^{N}((y_n + 1) - \mathbf{x}_n^\top(\mathbf{w} + e_0))^2 = \sum_{n=1}^{N}(y_n - \mathbf{x}_n^\top \mathbf{w})^2 \\
&\overset{(a)}{=} \sum_{n=1}^{N}(1 - y_n \mathbf{x}_n^\top \mathbf{w})^2
\end{aligned}
$$

- In step (a), we used $y_n^2 = 1$ (since $y_n \in \{\pm 1\}$).

Motivation: Deriving the MSE Loss

- For Least Squares, let's assume bias e_0 is component 0. Define $\tilde{\mathbf{w}} = \frac{1}{2}(\mathbf{w} + e_0)$ (this is a one-to-one mapping).
- The derivation becomes:

$$\begin{aligned} 4\sum_{n=1}^{N}(\tilde{y}_n - \mathbf{x}_n^\top \tilde{\mathbf{w}})^2 &= 4\sum_{n=1}^{N}(\frac{1}{2}(y_n+1) - \frac{1}{2}\mathbf{x}_n^\top(\mathbf{w}+e_0))^2 \\ &= \sum_{n=1}^{N}((y_n+1) - \mathbf{x}_n^\top(\mathbf{w}+e_0))^2 = \sum_{n=1}^{N}(y_n - \mathbf{x}_n^\top \mathbf{w})^2 \\ &\overset{(a)}{=} \sum_{n=1}^{N}(1 - y_n\mathbf{x}_n^\top \mathbf{w})^2 \end{aligned}$$

- In step (a), we used $y_n^2 = 1$ (since $y_n \in \{\pm 1\}$).
- This gives the MSE loss in margin form:

$$\text{MSE}(z, y) = (1 - yz)^2 \quad \text{where } z = \mathbf{x}_n^\top \mathbf{w}$$

Motivation: Deriving the Logistic Loss

- For Logistic Regression, we can also rewrite the loss:

$$\sum_{n=1}^{N} \log(1 + e^{\mathbf{x}_n^\top \mathbf{w}}) - \tilde{y}_n \mathbf{x}_n^\top \mathbf{w} = \sum_{n=1}^{N} \log(1 + e^{-y_n \mathbf{x}_n^\top \mathbf{w}})$$

Motivation: Deriving the Logistic Loss

- For Logistic Regression, we can also rewrite the loss:

$$\sum_{n=1}^{N} \log(1 + e^{\mathbf{x}_n^\top \mathbf{w}}) - \tilde{y}_n \mathbf{x}_n^\top \mathbf{w} = \sum_{n=1}^{N} \log(1 + e^{-y_n \mathbf{x}_n^\top \mathbf{w}})$$

- This is easily seen by checking the two cases:
  - $\tilde{y}_n = 1 \leftrightarrow y_n = 1$:

$$\log(1 + e^{\mathbf{w}^\top \mathbf{x}}) - \mathbf{w}^\top \mathbf{x} = \log(e^{\mathbf{w}^\top \mathbf{x}}(e^{-\mathbf{w}^\top \mathbf{x}} + 1)) - \mathbf{w}^\top \mathbf{x} \tag{4}$$

$$= \log(e^{\mathbf{w}^\top \mathbf{x}}) + \log(1 + e^{-\mathbf{w}^\top \mathbf{x}}) - \mathbf{w}^\top \mathbf{x} \tag{5}$$

$$= \log(1 + e^{-\mathbf{w}^\top \mathbf{x}}) \tag{6}$$

  - $\tilde{y}_n = 0 \leftrightarrow y_n = -1$:

$$\log(1 + e^{\mathbf{w}^\top \mathbf{x}}) - 0 = \log(1 + e^{-(-1)\mathbf{w}^\top \mathbf{x}}) \tag{7}$$

Motivation: Deriving the Logistic Loss

- For Logistic Regression, we can also rewrite the loss:

$$\sum_{n=1}^{N} \log(1 + e^{\mathbf{x}_n^\top \mathbf{w}}) - \tilde{y}_n \mathbf{x}_n^\top \mathbf{w} = \sum_{n=1}^{N} \log(1 + e^{-y_n \mathbf{x}_n^\top \mathbf{w}})$$

- This is easily seen by checking the two cases:
  - $\tilde{y}_n = 1 \leftrightarrow y_n = 1$:

$$\log(1 + e^{\mathbf{w}^\top \mathbf{x}}) - \mathbf{w}^\top \mathbf{x} = \log(e^{\mathbf{w}^\top \mathbf{x}}(e^{-\mathbf{w}^\top \mathbf{x}} + 1)) - \mathbf{w}^\top \mathbf{x} \tag{4}$$

$$= \log(e^{\mathbf{w}^\top \mathbf{x}}) + \log(1 + e^{-\mathbf{w}^\top \mathbf{x}}) - \mathbf{w}^\top \mathbf{x} \tag{5}$$

$$= \log(1 + e^{-\mathbf{w}^\top \mathbf{x}}) \tag{6}$$

  - $\tilde{y}_n = 0 \leftrightarrow y_n = -1$:

$$\log(1 + e^{\mathbf{w}^\top \mathbf{x}}) - 0 = \log(1 + e^{-(-1)\mathbf{w}^\top \mathbf{x}}) \tag{7}$$

- This gives the Logistic loss in margin form:

$$\text{LogisticLoss}(z, y) = \log(1 + \exp(-yz))$$